



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INFORMATION SYSTEMS

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

ALGORITHMS FOR AUTOMATED PLAY OF HACKEN-BUSH USING SURREAL NUMBERS

ALGORITMY STROJOVÉHO HRANÍ HACKENBUSHE S VYUŽITÍM SURREÁLNÍCH ČÍSEL

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

VÁCLAV MATYÁŠ

SUPERVISOR

VEDOUCÍ PRÁCE

doc. Ing. FRANTIŠEK ZBOŘIL, Ph.D.

BRNO 2026

Abstract

This bachelor's thesis deals with the design and implementation of two methods for converting a combinatorial game into algebraic form. The first method focuses on the specific game of Hackenbush, which contains red and blue edges. It converts a position in this game into a dyadic number by recursively traversing the possible moves of the left and right player. The second method converts a combinatorial game into a short game. This conversion is first demonstrated on the game of Hotpotch (Hackenbush with a green edge), after which the algorithm is generalised and an interface is proposed, which any combinatorial game can implement. By correctly implementing this interface, it is then possible to convert any combinatorial game into the form of a short game. For both methods, a graphical interface is subsequently developed, together with a short game module in Python.

Abstrakt

Tato bakalářská práce se zabývá návrhem a implementací dvou metod převodu kombinatorické hry do algebraické formy. První metoda se zaměřuje na specifickou hru Hackenbush, která obsahuje červené a modré hrany. Pozici v této hře převádí na dyadické číslo rekurzivním procházením možných tahů levého a pravého hráče. Druhá metoda převádí kombinatorickou hru na krátkou hru. Tento převod je nejprve demonstrován na hře Hotpotch (Hackenbush se zelenou hranou), algoritmus je poté zobecněn a je navrženo rozhraní, které může libovolná kombinatorická hra implementovat. Správnou implementací tohoto rozhraní je pak možné převést libovolnou kombinatorickou hru do formy krátké hry. Pro obě metody je následně vytvořeno grafické rozhraní, společně s modulem krátkých her v jazyce Python.

Keywords

combinatorial game theory, Hackenbush, Hotpotch, short games, canonical form, dyadic numbers, solver, memoization

Klíčová slova

kombinatorická teorie her, Hackenbush, Hotpotch, krátké hry, kanonická forma, dyadická čísla, řešič, memoizace

Reference

MATYÁŠ, Václav. *Algorithms for Automated Play of Hackenbush Using Surreal Numbers*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. Ing. František Zbořil, Ph.D.

Algorithms for Automated Play of Hackenbush Using Surreal Numbers

Declaration

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně pod vedením pana doc. Ing. Františka Zbořila, Ph.D. Uvedl jsem všechny literární prameny, publikace a další zdroje, ze kterých jsem čerpal. Pro jazykovou revizi technické zprávy jsem použil nástroj Claude. Při tvorbě a překladu komentářů v odevzdaném programovém kódu jsem využíval nástroj ChatGPT.

.....

Václav Matyáš

August 4, 2026

Acknowledgements

Rád bych poděkoval svému vedoucímu, panu doc. Ing. Františkovi Zbořilovi, Ph.D., za vstřícný přístup, cenné rady a za umožnění vypracovat vlastní téma, které pro něj bylo nové. Na závěr děkuji rodině a svým blízkým za podporu a povzbuzení po celou dobu studia.

Contents

1	Introduction	3
2	What Is a Combinatorial Game	4
3	Combinatorial Games	6
3.1	Hackenbush	6
3.2	Hotpotch	7
3.3	Nim	7
3.4	Informally Assigning Numbers to Hackenbush Positions	8
4	Short Games	10
4.1	Game Algebra	14
4.2	Infinitesimal Values	19
5	Dyadic Numbers	23
5.1	Definition of a Dyadic Number	23
6	Design and Implementation Using Dyadic Numbers	26
6.1	Data Representations of the Hackenbush Graph	26
7	Design and Implementation Using Short Games	32
7.1	From a Raw Game to a Short Game	32
7.2	Game Outcome	33
7.3	Explicit Recursive Conversion of a Raw Game	33
7.4	Memoization and Hash Tables	35
7.5	Canonicalization and Its Consequences	36
7.6	Splitting Hotpotch into Independent Components	38
7.7	Comparing Games: $G \geq H$	39
8	Generalizing the Short-Game Solver to Other Games	40
8.1	A More General Conversion from a Raw Game to a Short Game	41
8.2	Domineering	44
8.3	File Structure	45
9	Comparison of Approaches and Further Extensions	49
10	Conclusion	51
	Bibliography	52
A	Installation and Execution Guide	53

List of Figures

3.1	An example of a trivial Hackenbush position.	6
3.2	A Hotpotch position containing one green edge extending from the root.	7
3.3	A Nim position with heaps of sizes 6, 4, and 8 represented as Green Hackenbush. Each heap corresponds to one chain of green edges above the ground.	7
3.4	Three distinct basic Hackenbush positions.	8
3.5	The figure illustrates the process of determining the value of the position in Figure 3.5a by progressively adding other known values. A similar procedure can be found in [1, pp. 4–5].	9
4.1	Two positions, each violating at least one restricting condition of short games. . .	11
4.2	Decomposition of the position AB into independent components A and B , that is, $AB = A + B$	14
4.3	A visual example of converting a raw combinatorial game into a short game. . . .	16
4.4	A position with Blue to move, containing move 0 dominated by move 1.	17
4.5	A position with Blue to move, containing reversible move 1, reversible through move 0.	17
4.6	Decomposition of a game with one green edge extending from the ground. The blue arrow denotes Left's move to the zero game at the lower left; the red arrow analogously denotes Right's move to the zero game at the lower right.	19
4.7	A position with value $*$ $- 1$	20
4.8	A comparison of $*$ with positive dyadic numbers.	20
4.9	A comparison of $*$ with negative dyadic numbers.	20
4.10	Position a has a fuzzy value, so the player to move always wins. However, its sum with an identical position b is a win for Left.	21
4.11	A Hotpotch position with value $*$	21
4.12	Hotpotch positions with values $\uparrow$ (a) and $\downarrow$ (b).	22
5.1	A tree representation of dyadic numbers, where horizontal dashed lines indicate individual birthdays.	24
6.1	The Hackenbush data representation and a visual example with 5 vertices and 4 edges.	27
8.1	An example of an irregular Domineering position.	44
8.2	The directory and file tree, with the parts used by a newly implemented combinatorial game highlighted in green.	46
8.3	The Hotpotch graphical interface with explanations of the extensions.	47
8.4	The graphical interface of the short-game calculator.	48

Chapter 1

Introduction

Combinatorial game theory (CGT) is a mathematical discipline concerned with the analysis of two-player games with perfect information, in which the outcome depends only on the players' strategies, not on chance.

In the past, game analysis most often consisted of a fragmented study of diverse, unrelated games. At that time, it was not even possible to study more complex games, and the discipline did not advance significantly. This changed around 1930, when R. P. Sprague and P. M. Grundy introduced a more formal description of games. An important result of their work is the Sprague-Grundy theorem, because it shows that a broad class of games can be reduced to the game of Nim and can therefore be studied in a unified way. Subsequently, in 1976, John H. Conway published the book *On Numbers and Games*, in which he first formally described the treatment of games as we know it today. Conway showed that ordering, addition, and other arithmetic operations for game analysis can be formally defined over the structure of the possible moves of the first and second players. This thesis contains only a very small part of the possible analysis, because this is still a relatively young field.

Chapter 5 formally describes that some combinatorial games can be evaluated solely as dyadic numbers. One such game is Hackenbush, which is used to introduce these numbers, and Chapter 6 subsequently describes a method for converting such a Hackenbush game into a dyadic number.

The second method uses the framework of so-called short games. These are values expressed by the possible moves of the first and second players. This game form is more general and is used for Hotpotch, which requires this more general form for evaluating its value. This approach, explained in Chapter 8, can then be used to evaluate arbitrary additional combinatorial games.

Chapter 2

What Is a Combinatorial Game

There is no single correct collection of rules for a combinatorial game, and different variants can be found in the literature. For this thesis, the rules based on [5, 10] are sufficient. A combinatorial game is a game that satisfies the following rules.

1. There are two players, the Left player and the Right player, abbreviated simply as Left and Right.
2. Left and Right alternate moves.
3. Both players have all information, that is, complete information.
4. No move is affected by random events such as rolling dice or shuffling cards.
5. Under the normal-play convention, the player who can no longer make a move loses.
6. Both players are rational.

Each chapter will point out which rules are violated. The rules above, however, will always remain in force.

The rules that will be violated in some chapters include

7. the game contains a finite set of positions in which it can occur.
8. the game must always be able to end because one of the players can no longer make a move. This is also called the ending condition.

A classical example of a combinatorial game is Nim 3.3. It is a game with heaps of tokens, where each player removes any number of tokens from one heap. The outcome of any position can be determined using the nim-sum, as proved by Bouton in 1902 [2].

Every Nim position can be trivially converted into a Hotpotch position in which every edge is green. By creating a solver for any Hotpotch position, introduced in Section 3.2, I therefore also create a Nim solver. Conversely, if it can be verified that a given Hotpotch position corresponds to a Nim position, the efficient nim-sum rule can be used to solve that position.

Goals of Combinatorial Game Theory

Combinatorial game theory (CGT) addresses several goals. For me, the most important one will be to determine the outcome of the game for a given position, that is, to answer intuitively who will win. I will immediately extend this goal with the question of “how winning the position is.” More specifically, this text addresses the following goals:

- Find the exact outcome of a position in a given game, most often in the form of an algebraic expression.
- Determine the class to which a game position belongs.
- Understand the game from the perspective of combinatorial game theory and exploit its properties.

Outcome Classes

A natural question for every combinatorial game is which player wins under optimal play by both sides. Game positions can be divided into *outcome classes*. In general, there are four classes:

1. Positions in which the first player wins ($\mathcal{N}$, from *next*).
2. Positions in which the second player wins ($\mathcal{P}$, from *previous*).

For symmetric games such as Nim, there are only two classes. For asymmetric games, we consider two additional classes:

3. Positions in which Left wins regardless of move order ($\mathcal{L}$).
4. Positions in which Right wins regardless of move order ($\mathcal{R}$).

Traditionally, we say that a game is *solved* if there is an efficient rule for determining the outcome class of any position [10, p. 4]. In this thesis and in the implementation, this term means converting a game into its algebraic form, for which there is an efficient rule for determining the outcome class.

Chapter 3

Combinatorial Games

3.1 Hackenbush

Hackenbush is a combinatorial game in which two players alternate, Left and Right, corresponding to Blue and Red. Left moves by removing any blue edge together with all edges that are no longer connected to the ground. In the figures, the ground will most often be represented by a purple rectangle. Right moves analogously by removing a red edge [1, p. 2]. Hackenbush starts from an initial position containing blue and red edges. Each edge is connected either to another edge or to the ground. Under the normal-play convention, the player who can no longer make a move loses. There is also a misère variant, in which the player with no move wins.

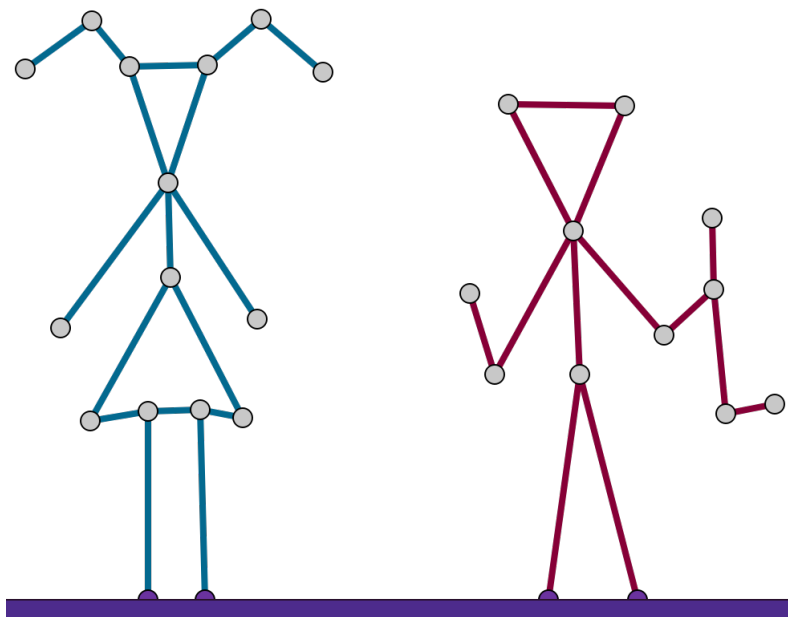


Figure 3.1: An example of a trivial Hackenbush position.

The position shown in Figure 3.1 contains two separate components that do not affect each other, which means that no move by the opposing player affects the choice of the next move. It is natural to reason that, because Left (Blue) and Right (Red) alternate moves, the player with more moves wins; in this case Blue, who has 17 moves, while Red has 13 moves. In a certain sense, the number $17 - 13 = 4$ could be assigned to this position.

3.2 Hotpotch

Hotpotch is a variant of classical Hackenbush. The game remains the same, but a new green type of edge is added. A green edge can be destroyed by either side, Left (Blue) or Right (Red). Blue edges can still be destroyed only by Left, and red edges only by Right [5, p. 188]. Moves available to both sides create values that cannot be expressed as dyadic numbers. This game will be used to introduce the general framework of short games.

One possible example of a Hotpotch position is a single green edge extending from the root (the ground).

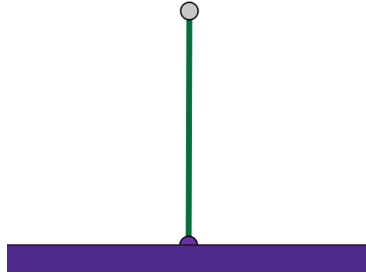


Figure 3.2: A Hotpotch position containing one green edge extending from the root.

3.3 Nim

Nim is a combinatorial game in which two players, Left and Right, alternate moves. The game starts from an initial position containing several heaps of tokens. On each turn, a player removes any number of tokens (at least one) from exactly one heap. Under the normal-play convention, the player who can no longer make a move loses, that is, the player whose turn begins with all heaps empty.

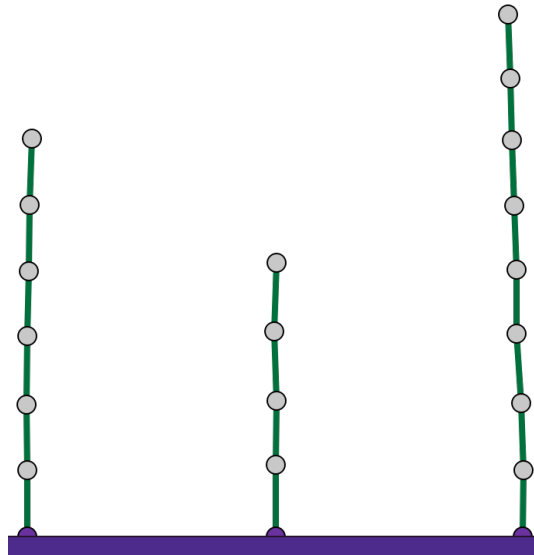


Figure 3.3: A Nim position with heaps of sizes 6, 4, and 8 represented as Green Hackenbush. Each heap corresponds to one chain of green edges above the ground.

The Nim position in Figure 3.3 can naturally be represented as a Hotpotch position [10, p. 180], where each heap of k tokens corresponds to a chain of k green edges above the ground.

Their equivalence could, for example, be proved using the Sprague-Grundy theorem. Removing tokens from a heap then corresponds to cutting an edge in the chain.

Solving a Nim Position

The outcome of any Nim position can be determined using the *nim-sum*. Let G be a Nim position containing heaps of sizes $a_1, a_2, \dots, a_n$. Then the nim-value of this position is

$$a_1 \oplus a_2 \oplus \dots \oplus a_n,$$

where $\oplus$ denotes the bitwise XOR operation.

Consider the basic empty Nim position, where $a_1 = a_2 = \dots = a_n = 0$. The nim-value of this position is 0. In this case, the second player wins.

Věta 3.1 (Bouton's theorem). *Let G be a Nim position. If G is a zero position, then every move from G leads to a nonzero position. If G is not a zero position, then there exists a move from G to a zero position.*

By induction on the number of remaining tokens, it can be proved that the player who maintains the invariant of handing a zero position to the opponent will eventually hand the opponent the zero position in which all heaps are empty, resulting in a loss for the player to move.

Detecting such a position in Hotpotch could optimize the search for a winning strategy.

3.4 Informally Assigning Numbers to Hackenbush Positions

Conway introduced the mathematical form of games rather early in *On Numbers and Games* [5]. It makes more sense to me first to build an intuition for what should be expected. Therefore, in this section I want to introduce informally the assignment of numbers, or values, to combinatorial games.

As an example, I will take the combinatorial game Hackenbush, attempt to determine who wins a particular position, and assign a meaningful number to that position.

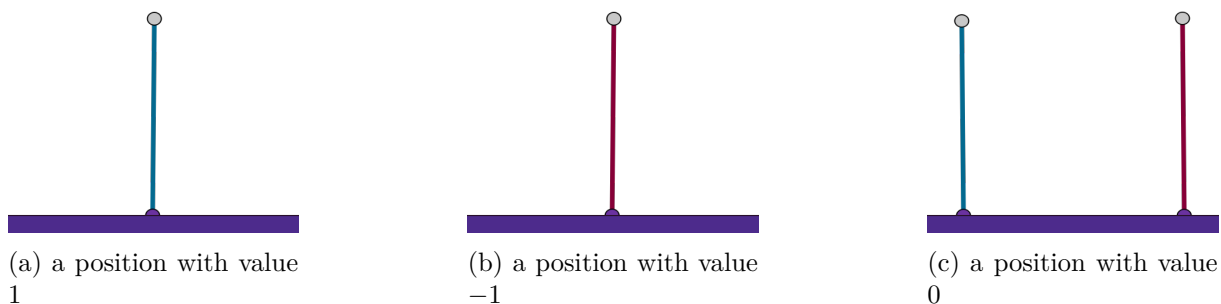


Figure 3.4: Three distinct basic Hackenbush positions.

Figure 3.4a shows a position with one blue edge. If Left (Blue) starts, Left destroys the blue edge, and Right (Red) cannot make any move, so Left wins under the normal-play convention. If Right starts, Right has no possible move and again loses under the normal-play convention. This is therefore a position of type $\mathcal{L}$. By a similar argument, the position with one red edge in Figure 3.4b is of type $\mathcal{R}$.

What about the position in Figure 3.4c? This position is simple enough to examine all possible moves from the perspective of Left and Right, or Blue and Red. In this position, the second player wins, so it is a position of type $\mathcal{P}$. Informally, the position is balanced.

The purpose of this summary is to show that Hackenbush contains positions that can be won unconditionally by Left or by Right, but not by both.

The most meaningful number for a position with one blue edge is 1 or -1 . Which of these values is chosen is arbitrary, but by convention one blue edge has the value 1. When a red edge is added to this blue edge (Figure 3.4c), the game changes from an unconditional win for Left to a win for the second player. If another red edge extending from the ground were added to the same game, the game would become an unconditional win for Right. One could say that somewhere between an unconditional win for Left and an unconditional win for Right lies a neutral value that favors neither player and is won by the second player. By convention, such a position is assigned the number 0.

Poznámka. In Hackenbush, there is no position in which the first player always wins. Such positions exist in Hotpotch.

Poznámka. The meaning of a *number* is described in Chapter 5. The present section provides only an informal introduction.

A Position with the Numerical Value of Half a Move

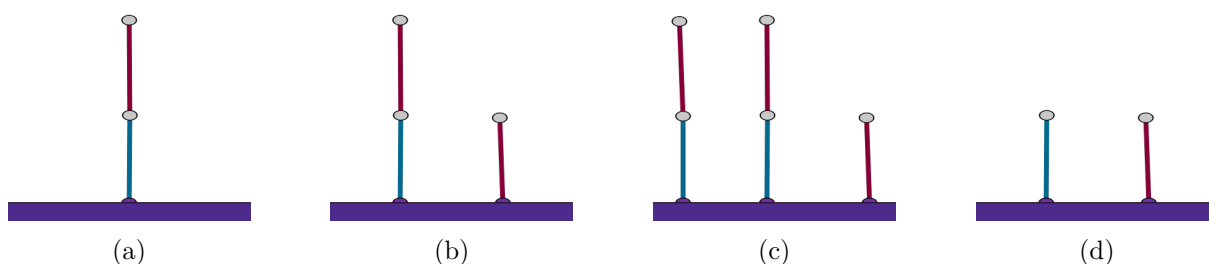


Figure 3.5: The figure illustrates the process of determining the value of the position in Figure 3.5a by progressively adding other known values. A similar procedure can be found in [1, pp. 4–5].

The classical method for determining a number without the framework of short games (see Chapter 4) is to add a known position to the unknown position. Consider Figure 3.5a; it can again be verified that this game is won unconditionally by Left (Blue). By adding one red edge, as in Figure 3.5b, the resulting game is instead won unconditionally by Right. The position 3.5a therefore does not have sufficient value to make the resulting position zero.

It can, however, be verified that the position in Figure 3.5c is already a zero position. If Left starts, Left destroys the blue edge, Right destroys the red edge on the blue edge, and the game reaches a known zero position in which the second player wins (Figure 3.5d), namely Right. If Right starts, Right destroys the red edge attached to the top of the blue edge; Left then destroys the blue edge together with the red edge on it, bringing the position to the known zero position (Figure 3.5d), so Left wins. Thus, two copies of this unknown position cancel a position with value -1 . Could one therefore say that the position in Figure 3.5a has value $\frac{1}{2}$? If we accept the assumption that positions can be added in this way (they can), then yes, this position has value $\frac{1}{2}$.

The convention is therefore that Left (Blue) wants to maximize the value of the position, while Right (Red) wants to minimize it.

The method in this section is insufficient, however. A formal framework is needed that assigns a precise value to every position and allows further operations on those values. This framework is provided by short games.

Chapter 4

Short Games

In the previous chapter, I assigned numbers to position values experimentally, by adding known positions and observing the outcome. Such a procedure cannot be used for proofs or implementation. A formal model is needed that assigns a value to each position directly from its structure. This model is given by short games.

Short games can generally express values of types other than the integers and rational numbers introduced in the previous section 3.4. Although such numbers can be introduced simply and informally, it makes more sense to present the more general form, short games, first.

Integers and dyadic numbers can then be constructed from short games. The meaning of a number will be introduced in Chapter 5.

Definition of a Short Game

Instead of describing a game using specific rules, for mathematical analysis it can be described abstractly, exclusively through the moves available to the players.

Definition 4.1 (Short game). *The following definition is taken from [10, pp. 53–54]. A short game G is an ordered pair $(\mathcal{G}^L, \mathcal{G}^R)$, where $\mathcal{G}^L$ and $\mathcal{G}^R$ are sets of “simpler” short games, the **Left** and **Right** options of the game G .*

I will often denote Left’s options in the game G by G^L and Right’s options by G^R . In simplified notation,

$$G = \{G^L \mid G^R\}$$

or, written out,

$$G = \{G^{L_1}, G^{L_2}, \dots \mid G^{R_1}, G^{R_2}, \dots\}.$$

*The base case of the recursion is the **zero** game $0 = \{\mid\}$, also called the **empty game** [5, 10].*

This is a simplified definition from which it may not be immediately apparent that a short game always has a finite number of elements. I therefore state the additional requirements for short games explicitly.

1. *Finiteness*: a short game has only finitely many subpositions (states that can be reached).
2. *Loopfreeness*: a short game cannot continue forever; there is no infinite sequence of moves.

The values of short games form a group $\mathbb{G}$ under game addition, whose identity element is $0 = \{\mid\}$ [10, p. 56].

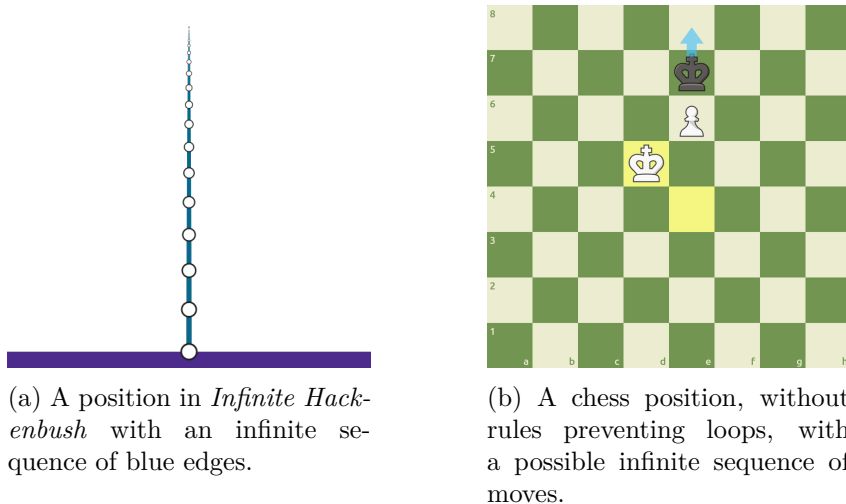


Figure 4.1: Two positions, each violating at least one restricting condition of short games.

Figure 4.1a shows a position in an extended version of Hackenbush in which a position may have infinitely many options. This position contains infinitely many blue edges, each extending from the preceding one. Such a position is not a short game because it violates the *finiteness* condition. It does not violate loopfreeness, however, because any move destroys the infinitely many remaining options. It is therefore a position that violates Rule 7.

Figure 4.1b shows a chess position without the *fifty-move rule*. If White attempted to win such a position, Black could, with an optimal strategy, cycle the game indefinitely. Such a position therefore violates the *loopfreeness* condition and thus violates Rule 8.

Poznámka. Chess is moreover not a combinatorial game because some positions can end in a draw, thereby violating Condition 5. It nevertheless makes sense to discuss chess because some methods can also be applied to games that may end in a draw, for example in the work of *Noam Elkies* [6].

Hackenbush, Hotpotch, and Nim satisfy these properties.

The definition is recursive, and a short game is defined in terms of simpler games. The recursion always terminates at the zero game 0, because short games are finite and loopfree.

Basic Terminology

Throughout the text I will often work with a short game of the form

$$G = \left\{ G^L \mid G^R \right\},$$

where G^L denotes the set of Left options and G^R the set of Right options. By $|G^L|$ I mean the number of elements in the set G^L , that is, the number of positions reachable by one move by Left. Analogously, $|G^R|$ denotes the number of positions reachable by one move by Right.

I will use the terms *game* and *position* according to context. The word *game* will most often denote the short game that is the main object under investigation at a given point. The word *position* will often denote a short game [5, par. 6, ch. 7] that, in the given context, occurs as an element of the option set of another game [9, p. 2]. For example, every element of G^L is itself a short game, provided that G is a short game, but relative to the game G I will call it a Left position or Left option.

The terms *game*, or *short game*, and *position* are interchangeable. The reason is that every position itself determines a short game that starts at that position. The distinction is purely stylistic.

Elements of the set of Left options will be denoted, for example, by

$$G^{L_1}, G^{L_2}, \dots$$

or generally by G^{L_i} . Analogously, elements of the set of Right options will be denoted by

$$G^{R_1}, G^{R_2}, \dots$$

or generally by G^{R_i} .

To denote a position obtained by a Right move from the Left option G^{L_i} , one could formally write, for example,

$$\left(G^{L_i}\right)^{R_j}.$$

For readability, however, I will use the abbreviated notation

$$G^{L_i R_j}.$$

This notation means that the i -th Left move is first made from the game G , and then the j -th Right move is made from the resulting position. Similarly, $G^{L_i R_j L_k}$ denotes the position reached by a sequence of a Left move, a Right move, and another Left move.

In *On Numbers and Games* [5], Conway freely interchanges the terms *game* and *short game* [5, pp. 71–72]. There are also long games, in which each set of positions may have infinitely many elements. Such games will not be discussed here. I will often state explicitly that an object is a short game; however, when the context makes it more natural to use only the term *game*, a short game is intended.

Here, to *solve a game* means to convert the raw game into the form of a short game.

I further assume that all definitions mentioned are inductive. Thus, for example, when I state something about an object x , I assume that the stated property has already been applied to the pair (x^L, x^R) . Conway uses the same method in [5], explaining it on page 16.

4.1 Game Algebra

Definition 4.2 (Negation from [5, p. 5]). *The negation of a game G is the game $-G$ defined recursively by*

$$-G = \left\{ -G^R \mid -G^L \right\}.$$

The game $-G$ is the same game with the players' roles, or equivalently the edge colors, exchanged.

For every short game G , $G + (-G) = 0$: the second player always wins $G + (-G)$ by copying every move of the opponent in the other component [1, pp. 33–34].

Games can be added. If several independent games are played simultaneously, for example several Hackenbush components, one of them is selected on each turn and a move is made in it.

Definition 4.3 (Sum from [5, p. 5]). *Let G and H be short games. Their disjunctive sum $G + H$ is defined recursively by*

$$G + H = \left\{ G^L + H, G + H^L \mid G^R + H, G + H^R \right\},$$

where $G^L + H$ denotes the set of all games of the form $G^{L_i} + H$, and analogously for the other terms.

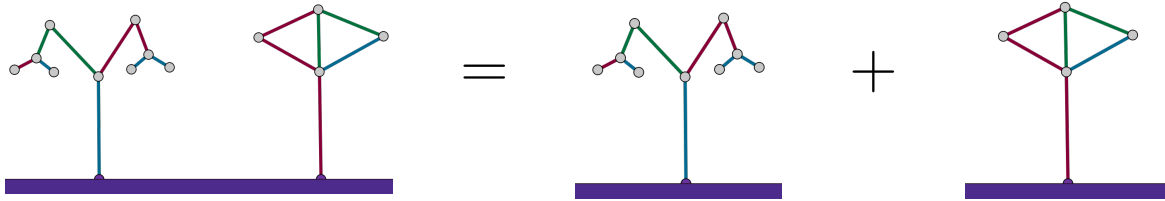


Figure 4.2: Decomposition of the position AB into independent components A and B , that is, $AB = A + B$.

It is worth noting that this decomposition into games that do not affect one another works in general even for different games. It is therefore possible, for example, to add a Hackenbush position to a Nim position.

All expected properties of addition remain valid, in particular:

Věta 4.1 (Properties of addition from [5, pp. 17–18]). *For arbitrary short games x , y , and z :*

1. $x + 0 \equiv x$,
2. $x + y \equiv y + x$,
3. $(x + y) + z \equiv x + (y + z)$,
4. $-(x + y) \equiv (-x) + (-y)$,
5. $-(-x) \equiv x$,
6. $x + (-x) = 0$.

We say that games x and y are equivalent, written $x \equiv y$, if their Left and Right sets are equivalent. This means that every Left option x^L is equivalent to some Left option y^L , every Right option x^R is equivalent to some Right option y^R , and conversely [5, pp. 15–16].

Poznámka. Line 6 is an equality rather than an equivalence.

Game Value

Definition 4.4 (Ordering, taken from [5]). $G \geq H$ if and only if, for every Right move G^R from G , $\neg(H \geq G^R)$ holds, and for every Left move H^L from H , $\neg(H^L \geq G)$ holds:

$$G \geq H \iff (\forall G^R : \neg(H \geq G^R)) \wedge (\forall H^L : \neg(H^L \geq G)).$$

Two games may look completely different and nevertheless have the same *value*, meaning that they behave identically in every context.

Definition 4.5 (Equality of games, taken from [5]). Games G and H are equal as values, written $G = H$, if $G \geq H$ and $H \geq G$.

The ordering $G \geq H$ expresses that the game G is at least as good for Left as H in any context.

An important observation is that equality is defined as a relation. Apparently different definitions can therefore determine the same number, so it is necessary to distinguish between the form $\{L \mid R\}$ of a number and the number itself.

All definitions are inductive. Thus, for example, to decide whether $x \leq y$, several similar questions must be considered for the pairs x^R, y and x, y^L . These questions are simpler than the original problem. At first sight, it may not be entirely clear that these inductions require no base case, because eventually they reduce to questions concerning elements of the empty set [5, pp. 4–5].

Assigning Short Games to Hotpotch Positions

Now that short games can be formally added and compared, it is useful to show how different Hotpotch positions appear as short games. The same procedure can be used for any combinatorial game.

A short game G has the form $\{G^L \mid G^R\}$, where the elements of the Left set denote positions available to Left and the elements of the Right set denote positions available to Right.

The simplest case is the zero game, in which neither player has a possible move. Such a game has the value $\{\mid\}$. As described in Section 3.4, this value equals 0, so $0 = \{\mid\}$. We say that such a position is zero [5, p. 75].

Consider Example 3.4a, which contains only one blue edge. If Left destroys this edge, a zero position is obtained; Right cannot move from the initial position. We write such a position as $\{0 \mid\}$. This is the canonical form of the value 1. Analogously, $2 = \{1 \mid\}$, $3 = \{2 \mid\}$, and for red edges $-1 = \{\mid 0\}$, $-2 = \{\mid -1\}$, and so on. In this way all integers can be constructed.

How should the position in Figure 3.5a, with a blue edge extending from the ground and a red edge on top of it, be written? Left destroys the blue edge and obtains the zero position; Right destroys the red edge and obtains the position 1. The notation is $\{0 \mid 1\}$.

Such a position behaves like the number $\frac{1}{2}$ because it is the simplest value between 0 and 1, namely the one with the earliest birthday, as illustrated in Figure 5.1. The precise meaning of *simplest* and the formal reason why game forms can be identified with dyadic rational numbers are explained in Chapter 5. I regard this assignment as a motivated observation.

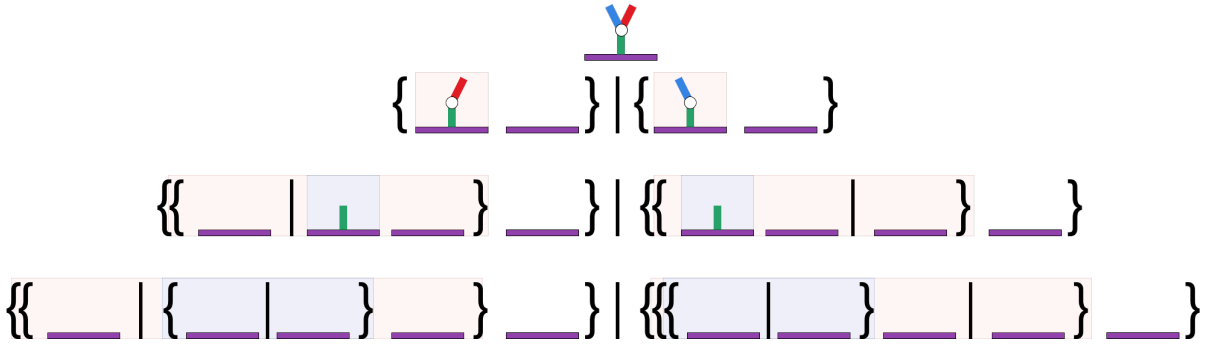


Figure 4.3: A visual example of converting a raw combinatorial game into a short game.

The same procedure can be applied to green edges: destroying a green edge is available to both players, so the corresponding position appears in the move sets of both players. The value in Figure 4.3 can be written as

$$\left\{ \{0 \mid \{0 \mid 0\}, 0\}, 0 \mid \{\{0 \mid 0\}, 0 \mid 0\}, 0 \right\}.$$

After canonicalization, this value simplifies to $\{0 \mid 0\}$. Such a game has no numerical value in the sense of dyadic numbers (see the section on dyadic numbers 5.1). The same result can also be obtained from a single green edge extending from the ground. These special positions are discussed in greater detail in the section on infinitesimal values 4.2.

Canonical Form

For example, $G = \{0, 0, 0, 0 \mid 2\}$ is a valid notation for a short game, but it is *not* in canonical form. A short game can be converted into canonical form by removing two types of moves: dominated and reversible moves [10, pp. 64–65]. I again remind the reader that all definitions are inductive, so if a property holds for an object x , I assume that it already holds, for example, for the pair (x^L, x^R) .

Dominated Moves

Suppose Left has, in a position G , a move to G^{L_i} and a move to G^{L_j} , with $G^{L_j} \geq G^{L_i}$, meaning that G^{L_j} is at least as good for Left as G^{L_i} . Left will never choose the move to G^{L_i} . In other words, Left always seeks the maximum value and Right always seeks the minimum value. Both players are rational, so by comparison they remove all of their worst positions, or possibly positions of equal value, because doing so does not change the outcome of the game.

Poznámka. After all dominated moves are removed, it is not generally the case that only one or no element remains in the set, because some values are incomparable, or confused.

Definition 4.6 (Dominated move, taken from [10, pp. 64–65]). *A Left move G^{L_i} is dominated by the move G^{L_j} if $G^{L_j} \geq G^{L_i}$. A Right move G^{R_i} is dominated by the move G^{R_j} if $G^{R_j} \leq G^{R_i}$.*

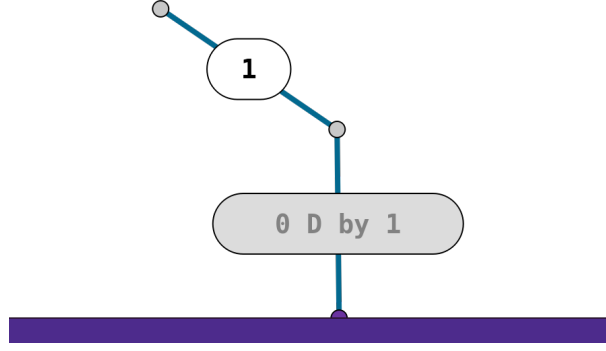


Figure 4.4: A position with Blue to move, containing move 0 dominated by move 1.

In Figure 4.4, there is a dominated move for Left, or Blue: more precisely, the move to edge 0 is dominated by the move to edge 1. If Left, or Blue, is to move, there are only two scenarios. Left destroys edge 0, obtaining a position with value 0. If Left destroys edge 1, the resulting position has value 1. Left wants to maximize the position's value and, being rational, will always choose the move to edge 1.

Reversible Moves

The second type of redundant move is more subtle. Left makes a move G^{L_i} , but Right has a reply $G^{L_i R_j}$ such that $G^{L_i R_j} \leq G$, so the resulting position is no better for Left than the original position G .

Definition 4.7 (Reversible move, taken from [10, pp. 64–65]). A Left move G^{L_i} is reversible through $G^{L_i R_j}$ if there exists a Right move $G^{L_i R_j}$ from the position G^{L_i} such that $G^{L_i R_j} \leq G$. Symmetrically, a Right move G^{R_i} is reversible through $G^{R_i L_j}$ if $G^{R_i L_j} \geq G$.

A reversible move cannot simply be removed; it must be *bypassed*. The move G^{L_i} is replaced directly by all moves $G^{L_1 R_1 L}$ that Left could make after Right reverses Left's move.

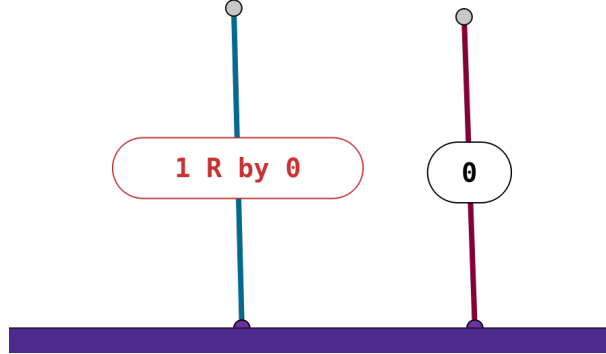


Figure 4.5: A position with Blue to move, containing reversible move 1, reversible through move 0.

Consider Example 4.5, in which there are two edges, one blue and one red, both extending from the ground. It is easy to verify that the value of this position is 0. The figure indicates that the move to the blue edge is reversible. Why? In simplified terms, a reversible move is one for which there exists a reply that returns the position to the *same* or a *worse* value. Thus, if Left, or Blue, destroys the blue edge, Right, or Red, must destroy the red edge and thereby return the position to the value 0. Since $0 \geq 0$, destroying the blue edge is reversible by destroying the red edge. Symbolically:

$$\begin{aligned} G = 0 &= \{-1 \mid 1\} = \{\{ \mid 0\} \mid \{0 \mid \}\}, \\ G^{L_1} &= \{ \mid 0\} \\ G^{L_1 R_1} &= 0 \\ G^{L_1 R_1} &\geq G \end{aligned}$$

Once this has been verified, the move G^{L_i} can be replaced by the moves $G^{L_1 R_1 L}$, which in this case are Left's moves in the zero position, namely none. This gives

$$G = \{ \mid \{0 \mid \} \}$$

It can be observed that Figure 4.5 is somewhat misleading, because the same reasoning shows that Right's move is also reversible. This is caused by the created application, which displays dominated and reversible moves only for the player whose turn it is. In the same way, one can verify that after removing all reversible moves one obtains

$$\{\{ \mid 0\} \mid \{0 \mid \}\} = \{ \mid \}.$$

It is worth mentioning that removing dominated moves and bypassing reversible moves *does not change* the value of the game [5, Theorem 68, pp. 110–111].

Existence and Uniqueness of Canonical Form

Definition 4.8 (Canonical form, taken from [10, p. 66]). *A short game K is in canonical form if none of its subpositions contains dominated or reversible moves.*

For every short game G , there exists exactly one canonical form $K = G$. This form is obtained by repeatedly removing dominated moves and replacing reversible moves with simpler equivalent options until no further such modifications are possible [5, Theorem 69, p. 112].

4.2 Infinitesimal Values

Dyadic numbers can be understood as a special case of short-game values. Some game positions, however, cannot be expressed by a dyadic number. Their value must be described by a more general short game. Such values include *infinitesimal* values.

Every Hackenbush position can be expressed as a dyadic number [4], which can be compared, added, and subjected to other simple operations directly. The full framework of short games must be used to solve Hotpotch precisely because Hotpotch contains infinitesimal values and their sums, which must be compared by definition, unlike dyadic numbers.

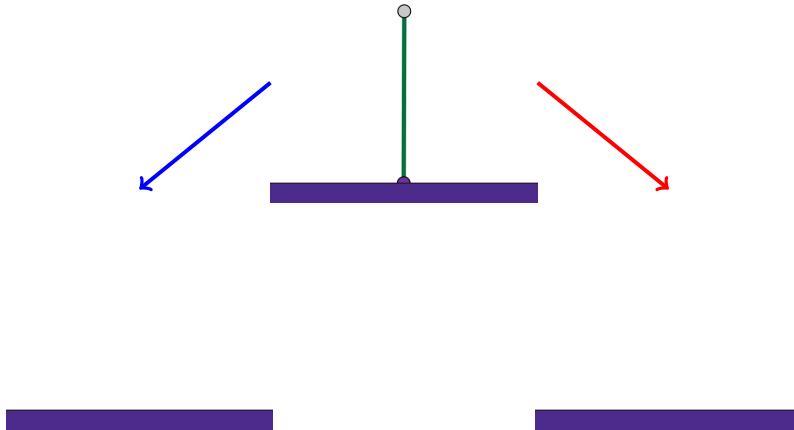


Figure 4.6: Decomposition of a game with one green edge extending from the ground. The blue arrow denotes Left's move to the zero game at the lower left; the red arrow analogously denotes Right's move to the zero game at the lower right.

An example of a game with an infinitesimal value is shown in Figure 4.6, which depicts a game with one green edge extending from the root vertex, or ground. Either player can destroy the green edge. If Left starts, Left destroys the green edge and leaves the opponent in a position from which no move can be made, so the opponent loses under the normal-play convention. If Right starts, Right analogously destroys the green edge and wins. All possible continuations have therefore been considered, and the first player wins. The first instinct may be to compare this with the zero position, but the first player loses the zero position because no move is available. The value in Figure 4.6 is therefore neither zero nor any dyadic number.

By rewriting the game as a short game, it can be assigned the value $\{0 \mid 0\}$, which is given the special symbol $*$.

The next question is how such a game compares with other values. Relations are commonly investigated by adding another position to the game and observing how the total value changes, ideally so that the sum becomes zero. In this way one can determine the value of the expression being sought, or at least its relationship to other values.

Consider the position shown in Figure 4.7.

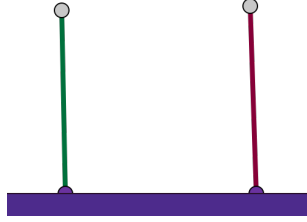


Figure 4.7: A position with value $* - 1$.

This game has value $* + (-1)$. It is easy to verify that the position is a win for Red, so $* - 1 < 0$, and hence $* < 1$.

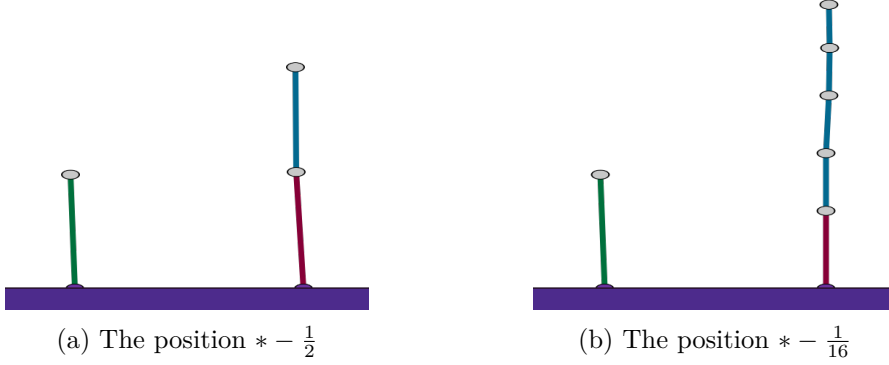


Figure 4.8: A comparison of $*$ with positive dyadic numbers.

Likewise, Figure 4.8 verifies that $* < \frac{1}{2}$ and $* < \frac{1}{16}$, and in general $* < \frac{1}{2^k}$ for every natural number k . Analogously, one can verify that $* > -1$, $* > -\frac{1}{2}$, $* > -\frac{1}{16}$, and in general $* > -\frac{1}{2^k}$ for every natural number k , as illustrated in Figure 4.9.

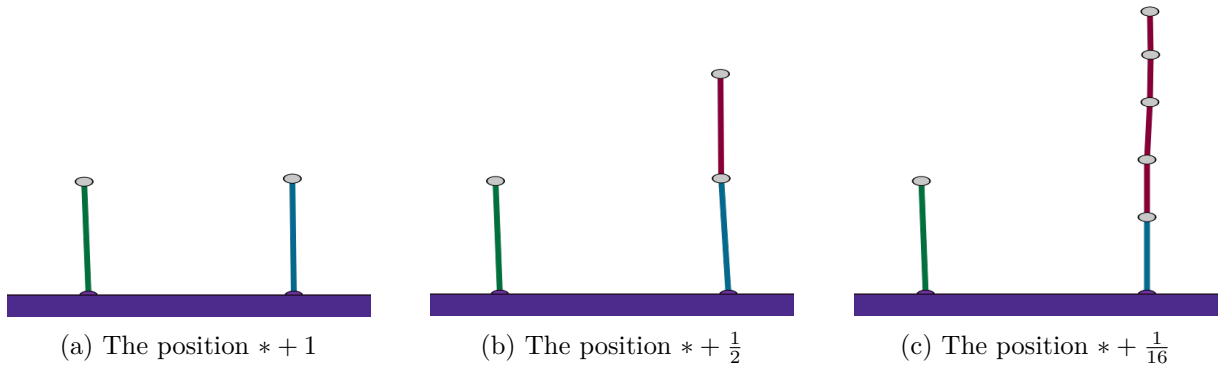


Figure 4.9: A comparison of $*$ with negative dyadic numbers.

Together, therefore,

$$-\frac{1}{2^k} < * < \frac{1}{2^k} \quad \text{for every } k \geq 0.$$

The only number introduced so far that satisfies this condition is 0, but as shown above, $* \neq 0$. The relation between $*$ and zero is called *incomparability* [1, pp. 29–30], or being *confused with*: we write $* \parallel 0$. Games incomparable with zero, $G \parallel 0$, are called *fuzzy*. Such games have the property that the first player wins [1, p. 28].

The value $*$ can be added, subtracted, and compared just like any other game value. The sum of values G and $*$ is written $G + *$, or in abbreviated form $G*$; this must not be confused with $*G$ [1, p. 39].

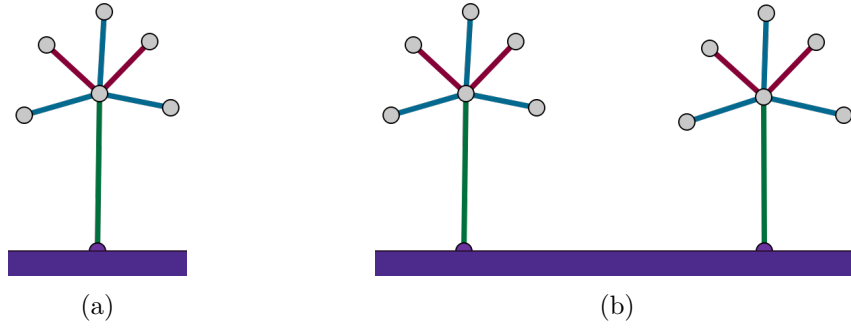


Figure 4.10: Position **a** has a fuzzy value, so the player to move always wins. However, its sum with an identical position **b** is a win for Left.

At first sight, it might seem that the position with a fuzzy value shown in Figure 4.10a favors neither player and should therefore be assigned the value 0. Such a convention would be misleading, however, because fuzzy positions often favor one player even though either player can win when moving first. As Figure 4.10b shows, the sum of two fuzzy values can be unambiguously favorable to one player.

Star. The game $*$ = $\{0 \mid 0\}$ is fuzzy ($* \parallel 0$) and is its own negation: $* + * = 0$ [1, p. 66]. In the context of Hackenbush, $*$ corresponds to one green edge above the ground.

Definition 4.9 (Nimbers). *Nimbers are defined recursively by*

$$*0 = 0, \quad *1 = * \quad *n = \{ *0, *1, \dots, *(n-1) \mid *0, *1, \dots, *(n-1) \}.$$

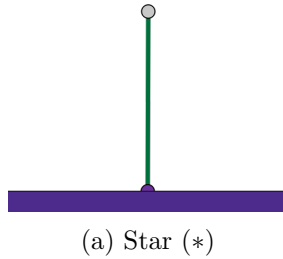


Figure 4.11: A Hotpotch position with value $*$.

Nimbers are the values of Green Hackenbush positions and, more generally, impartial games. A chain of k green edges above the ground has value $*k$.

Up and Down. Conway introduced further infinitesimal values that are not numbers. The motivation was to describe positions in which one player has a definite advantage, but the advantage is so small that adding any positive dyadic number, however small, eliminates it, a situation typical of Hotpotch.

Definition 4.10 (Up and Down).

$$\uparrow = \{0 \mid *\}, \quad \downarrow = \{*\mid 0\} = -\uparrow.$$

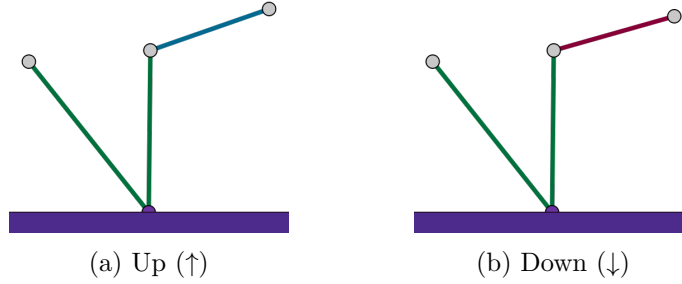


Figure 4.12: Hotpotch positions with values $\uparrow$ (a) and $\downarrow$ (b).

Why is $\uparrow > 0$? Left can move to 0, which is a loss for the player to move and therefore a win for the second player, whereas Right can move only to $*$, a fuzzy position won by the first player. Right therefore has no good reply, and $\uparrow$ is positive. At the same time, $\uparrow < \frac{1}{n}$ for every $n > 0$. Thus $\uparrow$ is positive but smaller than every positive dyadic number. It is therefore an infinitesimal of the same general character as $*$, but this time a nonzero value favoring Left.

We have $\uparrow \parallel *$, so they are incomparable, but $\uparrow + \uparrow > *$.

Definition 4.11 (Infinitesimal game, according to [10]). *A short game G is infinitesimal if $x > G > -x$ for every dyadic number $x > 0$.*

The developed library can recognize these basic special values and assign them the correct special symbols. However, the algorithm for converting a short game into a symbolic representation has not been formally developed, so it cannot in general convert a short game into the simplest symbolic form. The graphical interface also contains a short-game calculator that uses additional functions from the developed library and can perform a basic conversion from a symbolic short-game representation to the internal short-game data structure.

An algorithm for deciding whether a short game is infinitesimal is not immediately apparent from the definition. The library therefore also contains a function that uses the method described in [3, Theorem 4, p. 413] to determine whether a given short game is infinitesimal. This function can be used in the Hotpotch graphical-interface calculator.

Chapter 5

Dyadic Numbers

John Conway was not the first person to use the term *surreal numbers*. It had already appeared in the book *Surreal Numbers* [7], written by Donald E. Knuth in 1974.

At the time, Conway was working with Elwyn Berlekamp and Richard Guy on the book *Winning Ways*. The collaboration was not simple: Berlekamp's formal style clashed with Conway's more playful approach, and disputes arose over the structure of the book. Berlekamp also felt that an emphasis on transfinite numbers would substantially narrow the readership. In 1975, Conway therefore decided to write *On Numbers and Games* by himself, in a single week [10, p. 487].

It can be observed that this chapter does not discuss *surreal numbers*. The reason is that *surreal numbers* form a class that is far too general [10, p. 401] for solving the problems considered in this thesis.

What Are Surreal Numbers?

To understand why, it is first necessary to understand what *surreal numbers* are. A **long game** $x = \{x^L \mid x^R\}$ is a *surreal number* if $\forall l \in x^L, \forall r \in x^R : l < r$ [10, p. 401].

It is not appropriate to call this explanation of surreal numbers a definition, because it is not formally clear here what long games are.

Long games may in general have *infinitely* many possible positions [10, p. 398]. This thesis does not use, and cannot use, such values because some practical properties of short games do not hold in general for long games, and therefore do not hold for surreal numbers either.

The reason this chapter includes a brief section on surreal numbers is to prevent the reader from being confused by terminology in other sources.

Besides short games, this thesis uses a second type of value called *dyadic numbers*.

5.1 Definition of a Dyadic Number

Definition 5.1 (Dyadic number). *A rational number is dyadic if its denominator, in lowest terms, is a power of two, including the denominator $2^0 = 1$. The set of dyadic numbers is*

$$\mathbb{D} = \left\{ \frac{m}{2^n} : m \in \mathbb{Z}, n \in \mathbb{N}_0 \right\}.$$

A short game $G = \{G^L \mid G^R\}$ is a **number** if $\forall l \in G^L, \forall r \in G^R : l < r$ [10, Definition 3.1, p. 69]. Such a game behaves like a number in the sense that, in simplified form, “ $G^L < G < G^R$ ” [10, Proposition 3.2, p. 69]. Thus, the game value lies strictly between its Left and Right options.

As Siegel states [10, p. 70], the set of all such short games forms a subgroup of the group $(\mathbb{G}, +)$ that is isomorphic to $\mathbb{D}$. For this reason, this subgroup can be identified with the corresponding dyadic number, and one may write $\frac{m}{2^n}$ directly instead of the game value. This subgroup is simply denoted by $\mathbb{D}$.

The most important difference between numbers and short games is that numbers, or dyadic numbers [10, p. 70], are linearly ordered [5, p. 15].

In *On Numbers and Games*, where Conway introduced a more formal description of games, he used the term “number” rather than the now familiar term surreal number. D. E. Knuth had introduced the name *surreal numbers* three years earlier, but these concepts were still new at the time, and Conway did not use the expression in his first book.

Birthday

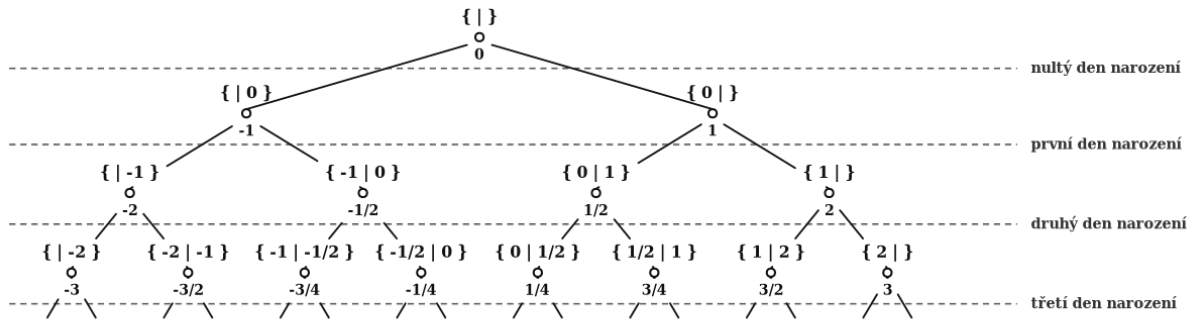


Figure 5.1: A tree representation of dyadic numbers, where horizontal dashed lines indicate individual birthdays.

Definice 5.2 (Birthday). *The birthday of a game G , denoted $b(G)$, is the minimum number of recursive steps required to construct G from the zero game 0. We say that G was born on day n if $b(G) = n$.*

Příklad 5.1 (Birthday of basic games).

$$\begin{array}{ll}
 b(0) = 0 & \text{base of the recursion} \\
 b(1) = b(-1) = 1 & \text{constructible from } \{0\} \\
 b(\frac{1}{2}) = 2 & \text{requires 1 from day 1}
 \end{array}$$

The birthday corresponds to the order in which dyadic numbers are constructed. Every number is created from already existing values, and its birthday is one greater than the maximum birthday of its options:

$$b(x) = \max(1 + b(L), 1 + b(R)).$$

Simplicity Rule

For a numerical short game

$$G = \left\{ G^L \mid G^R \right\}$$

its value is the simplest number greater than all Left options and simultaneously smaller than all Right options [5, p. 22]. Thus, we seek the simplest number x satisfying

$$\forall l \in G^L, \forall r \in G^R : \quad l < x < r.$$

Here, the simplest number means the number with the earliest birthday.

Příklad 5.2.

$$\begin{aligned} \{0 \mid 1\} &= \frac{1}{2}, & \{1 \mid 2\} &= \frac{3}{2}, & \left\{ 2 \mid 2\frac{1}{2} \right\} &= 2\frac{1}{4}, & \left\{ -3 \mid -3\frac{1}{2} \right\} &= -3\frac{1}{4}, \\ \{2 \mid 4\} &= 3, & \{-1 \mid 1\} &= 0, & \{-2 \mid 0\} &= -1, & \{-1 \mid 2\} &= 0, \\ \{2 \mid \} &= 3, & \{-100 \mid \} &= 0, & \left\{ \mid \frac{5}{2} \right\} &= 2, & \{ \mid -2 \} &= -3. \end{aligned}$$

Consider the example $\{-1 \mid 2\} = 0$. Between -1 and 2 lie infinitely many possible numbers, such as $-\frac{1}{2}$ or $1\frac{1}{4}$, but all of them have a later birthday than 0 (see Figure 5.1), which also satisfies $-1 < 0 < 2$.

All numbers in Examples 5.2 are constructed in this way.

Chapter 6

Design and Implementation Using Dyadic Numbers

This chapter concerns only the numerical case corresponding to Hackenbush. As stated in the previous chapter, finite positions of this variant have values in the set of dyadic rational numbers, and therefore do not need to be represented as general short games.

The objective of this part is not to develop a general solver for the Red-Blue-Green variant of Hackenbush, also called Hotpotch, but to demonstrate another possible method for solving this problem.

6.1 Data Representations of the Hackenbush Graph

The only dynamic operation while traversing moves is edge deletion. This property allows the description of the graph structure to remain entirely static. Information about whether an edge is live, that is, not yet deleted, is encoded as an active bit in an n -bit integer, hereafter denoted by the type `edge_mask_t`. The bit at position k corresponds to the edge with index k ; removing an edge is therefore a single bit operation.

Here I introduce the term *raw game*. A raw game means any data representation of a game that is *not expressed* using the data representation of a short game. The short-game data representation is described in Section 7.1. A raw Hackenbush game is represented by the `BaseGraph` structure, which stores only immutable information about edge connectivity, the number of edges, and the number of vertices:

```

1 // src/dyadics/dyadics/C/raw_game.h
2 // src/short-games/C/games/hotpotch/hotpotch.h
3 typedef struct {
4     uint8_t u;
5     uint8_t v;
6     EdgeColor color;
7 } Edge;
8
9 typedef struct {
10    uint8_t num_vertices;
11    uint8_t num_edges;
12    Edge edges[MAX_EDGES];
13 } BaseGraph;

```

Listing 6.1: Data representation of a Hackenbush graph

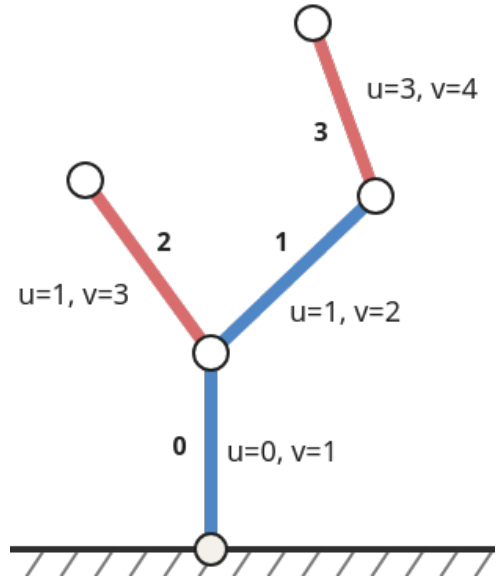


Figure 6.1: The Hackenbush data representation and a visual example with 5 vertices and 4 edges.

This method expresses the values of Hackenbush positions solely by dyadic numbers,

$$\frac{m}{2^n},$$

where $m \in \mathbb{Z}$, $n \in \mathbb{N}_0$. This representation permits comparison and construction of new values. The approach is used in `src/dyadics/C/dyadic.h`, where the following structure is defined:

```

1 // src/dyadics/C/dyadics.h
2 // G = num / 2^exp
3 typedef struct {
4     long long num;
5     int exp;
6 } Dyadic;

```

Listing 6.2: Dyadic-number structure.

6.1.1 The Simplicity Rule - the Simplest Dyadic Number Between Dyadic Numbers

Suppose the possible Left and Right positions have already been expressed inductively as dyadic numbers D_L and D_R , where $D_L, D_R \in \mathbb{D}$. An equivalent formulation using a short game is $\{D_L \mid D_R\}$. Short-game notation is not needed, however, because the value $\{D_L \mid D_R\}$ can be expressed as the simplest dyadic number $D \in \mathbb{D}$ such that “ $D_L < D < D_R$.” The implementation requires $|D_L|, |D_R| \in \{0, 1\}$; according to the number of elements, Algorithms 1, 2, and 3 are then used.

This condition can be achieved precisely because dyadic numbers are linearly ordered, as stated in Section 5.1. This guarantees that a set of dyadic numbers always contains a minimum and a maximum element. While traversing the Left options, it is therefore sufficient to retain only one element, L_{max} , and analogously only R_{min} while traversing the Right options, and to use them to compute the dyadic number of the position. In this way, the tree structure of short games can be avoided and the entire process simplified.

Description of Finding the Simplest Dyadic Number

Finding the simplest dyadic number between two dyadic numbers is not trivial, so this section derives an appropriate algorithm.

Let $a < b$ be dyadic numbers. We seek the simplest dyadic number x such that

$$a < x < b.$$

Here, *simplest* means the dyadic number with the earliest possible birthday. For dyadic rational numbers, this is equivalent to requiring that, after reduction, the number has the form

$$x = \frac{n}{2^k}$$

with the smallest possible exponent k . For a given k , we want to determine whether some multiple of $\frac{1}{2^k}$ lies strictly between a and b . This means finding an integer n satisfying

$$a < \frac{n}{2^k} < b.$$

One way to derive a general algorithm is to compare successively the nearest possible values to a and b at precision $\frac{1}{2^k}$. Rounding a number x down to the nearest multiple of $\frac{1}{2^k}$ can be written as

$$\frac{\lfloor x \cdot 2^k \rfloor}{2^k}.$$

The smallest multiple of $\frac{1}{2^k}$ strictly greater than a is $n_{\min}$, and the largest multiple of $\frac{1}{2^k}$ strictly smaller than b is $n_{\max}$, where

$$n_{\min} = \frac{\lfloor a \cdot 2^k \rfloor + 1}{2^k}, \quad n_{\max} = \frac{\lceil b \cdot 2^k \rceil - 1}{2^k}.$$

Poznámka. The function $\lfloor \cdot \rfloor$ always denotes rounding toward $-\infty$, that is, the floor of the number. Conversely, $\lceil \cdot \rceil$ always denotes rounding toward $+\infty$, that is, the ceiling of the number.

The values $n_{\min}$ and $n_{\max}$ define the bounds of the interval being searched. A candidate exists if

$$n_{\min} \leq n_{\max}.$$

If this condition holds, then the simplest dyadic number strictly greater than a and strictly smaller than b is $n_{\min}$.

The algorithm can then be simplified by postponing division by the denominator, because

$$\begin{aligned} n_{\min} &\leq n_{\max} \\ \frac{\lfloor a \cdot 2^k \rfloor + 1}{2^k} &\leq \frac{\lceil b \cdot 2^k \rceil - 1}{2^k} \\ \lfloor a \cdot 2^k \rfloor + 1 &\leq \lceil b \cdot 2^k \rceil - 1. \end{aligned}$$

Because the values $k = 0, 1, 2, \dots$ are examined in increasing order, the first solution found has the smallest possible denominator and is therefore the simplest. The complete procedure is summarized in Algorithm 1.

Algorithm 1 The simplest dyadic number between two values

Require: Dyadic numbers a, b , where $a < b$

Ensure: A dyadic number x satisfying $a < x < b$

```

1: for  $k = 0, 1, 2, \dots$  do
2:   Determine the integer interval
      $n_{\min} = \lfloor a \cdot 2^k \rfloor + 1, \quad n_{\max} = \lceil b \cdot 2^k \rceil - 1$ 
3:   if  $n_{\min} \leq n_{\max}$  then
4:     return  $x = \frac{n_{\min}}{2^k}$ 
5:   end if
6: end for
```

The simplest dyadic number above a value

If only a Left option exists, the simplest dyadic number greater than the given value is sought.

Algorithm 2 The simplest dyadic number above a value

Require: A dyadic number a

Ensure: A dyadic number x satisfying $x > a$

```
1: Determine the integer  $q = \lceil a \rceil$ 
2: if  $q > a$  then
3:   return  $q$ 
4: else
5:   return  $q + 1$ 
6: end if
```

The simplest dyadic number below a value

Analogously, if only Right options exist, the simplest dyadic number smaller than the given value is sought.

Algorithm 3 The simplest dyadic number below a value

Require: A dyadic number b

Ensure: A dyadic number x satisfying $x < b$

```
1: Determine the integer  $q = \lfloor b \rfloor$ 
2: if  $q < b$  then
3:   return  $q$ 
4: else
5:   return  $q - 1$ 
6: end if
```

These three operations correspond to the definition of a dyadic-number value and are used when computing position values in the solver.

Converting a Raw Game to a Dyadic Number

Hackenbush has the property that the value of every position is a dyadic number. It is therefore sufficient to convert the raw game into a dyadic number and compare the result with zero.

The following recursive algorithm converts a raw game into a dyadic number.

Algorithm 4 Recursive computation of a Hackenbush position's value

Require: Graph g , mask of live edges $live_mask$

Ensure: The dyadic value of the position

```
1: if  $live\_mask \in memo$  then
2:   return  $memo[live\_mask]$ 
3: end if
4: if  $live\_mask = 0$  then
5:   return 0
6: end if
7:  $has\_left \leftarrow \text{false}$ ,  $has\_right \leftarrow \text{false}$ 
8: for each edge  $e \in live\_mask$  do
9:    $child \leftarrow \text{cleanup}(g, live\_mask \setminus \{e\})$ 
10:   $v \leftarrow \text{solve}(g, child)$ 
11:  if edge  $e$  is blue then
12:    if  $\neg has\_left$  or  $v > L_{\max}$  then  $L_{\max} \leftarrow v$ ;  $has\_left \leftarrow \text{true}$ 
13:    end if
14:  else
15:    if  $\neg has\_right$  or  $v < R_{\min}$  then  $R_{\min} \leftarrow v$ ;  $has\_right \leftarrow \text{true}$ 
16:    end if
17:  end if
18: end for
19: if  $\neg has\_left \wedge \neg has\_right$  then  $result \leftarrow 0$ 
20: elseif  $has\_left \wedge \neg has\_right$  then  $result \leftarrow \text{simplestAbove}(L_{\max})$ 
21: elseif  $\neg has\_left \wedge has\_right$  then  $result \leftarrow \text{simplestBelow}(R_{\min})$ 
22: elseif  $L_{\max} < R_{\min}$  then  $result \leftarrow \text{simplestBetween}(L_{\max}, R_{\min})$ 
23:  $memo[live\_mask] \leftarrow result$ 
24: return  $result$ 
```

The function first recursively solves all positions reachable in one move and constructs the value of the current position from their values. The recursion depth is bounded above by the number of edges, because every move removes exactly one edge and the game is finite.

The computation begins by checking the hash table *memo*. If the position is not found, the algorithm checks whether the live mask is zero. This corresponds to the zero game and also serves as the principal termination condition of the recursion.

The algorithm then iterates over all live edges (line 8). For each edge, a new position is created by removing it and its value is computed recursively. At the same time, the aggregators $L_{\max}$ and $R_{\min}$ are maintained throughout the loop (lines 11 and 14), which implicitly removes dominated moves.

Chapter 7

Design and Implementation Using Short Games

The preceding chapters introduced two perspectives on the value of a position. In the numerical Hackenbush case, it is sufficient to work with dyadic rational numbers, but general Hotpotch requires a full representation of short games. This chapter describes how this general framework is translated into a solver implementation.

7.1 From a Raw Game to a Short Game

To determine the outcome of a game, the raw game must be converted into the form of a short game. The definition of a short game implies a recursive structure: a game is described by a set of simpler games, which are again described by sets of still simpler games, until a game is reached in which no move can be made. In Hotpotch, these are positions with bit mask 0.

A natural representation is therefore to store, for every position, pointers to its Left and Right options. Each position contains only information about its simpler games. Except in positions that contain no further games, it does not even make sense to store information about the position using a bit mask. At leaf nodes, one could store the information that no further move can be made, but this information can also be obtained from *L_count* and *R_count*: if *L_count* = *R_count* = 0, the position is the zero game, making any additional indication redundant. It can therefore be observed that a game represented as a short game *does not contain* information about the internal structure of the original game.

In the implementation, a short game is represented by the **Game** structure:

```
1 // src/short-games/C/short_game.h
2 typedef struct Game {
3     int L_count;
4     int R_count;
5     struct Game **left; /* Array of pointers to Left options. */
6     struct Game **right; /* Array of pointers to Right options. */
7 } Game;
```

Listing 7.1: Short-game node structure.

The raw Hotpotch representation remains the same as the Hackenbush representation in Listing 6.1; only a new edge-color option has been added.

7.2 Game Outcome

The outcome of a game G determines which player wins under optimal play by both sides. There are four outcome classes depending on the relation of G to zero. Importantly, all four classes can be expressed exclusively using the comparison $\geq$, which is available in the solver.

Definice 7.1 (Game outcome). *Let G be a short game and $0 = \{ \mid \}$ the zero game. Then*

$$\begin{aligned} G > 0 &\iff G \in \mathcal{L} \iff G \geq 0 \wedge \neg(0 \geq G), \\ G < 0 &\iff G \in \mathcal{R} \iff 0 \geq G \wedge \neg(G \geq 0), \\ G = 0 &\iff G \in \mathcal{P} \iff G \geq 0 \wedge 0 \geq G, \\ G \parallel 0 &\iff G \in \mathcal{N} \iff \neg(G \geq 0) \wedge \neg(0 \geq G). \end{aligned}$$

It is therefore sufficient to perform two comparisons: $G \geq 0$ and $0 \geq G$. Their combination uniquely determines the outcome class:

Table 7.1: Determining the outcome class from two calls to $\geq$.

$G \geq 0$	$0 \geq G$	Class	Outcome
1	0	$\mathcal{L}$	Left wins regardless of move order
0	1	$\mathcal{R}$	Right wins regardless of move order
1	1	$\mathcal{P}$	the second player wins
0	0	$\mathcal{N}$	the first player wins

In the solver implementation, the game outcome is determined in exactly this way, using two calls to `game_geq` with the value 0 as the reference point, without explicitly computing a winning strategy.

7.3 Explicit Recursive Conversion of a Raw Game

The previous section 7.2 described the idea for solving a raw game: convert the raw game into the mathematical form of a short game and compare it with zero twice. How, then, can a raw game be converted into a short game? My first attempt used an explicit method, meaning that the conversion algorithm contains elements specific to Hotpotch, such as the `cleanup` function, which removes all edges no longer connected to the ground. This approach is less general, but it makes the subsequent solution easier to understand.

Algorithm 5 Recursive computation of a component as a short game

Require: Graph g , mask of live edges $live_mask$

Ensure: A pointer to the short game G

```
1: if  $(g, live\_mask) \in position\_cache$  then
2:   return  $position\_cache[g, live\_mask]$ 
3: end if
4: Initialize arrays  $L \leftarrow []$ ,  $R \leftarrow []$ 
5: for each live edge  $e \in live\_mask$  do
6:    $child\_mask \leftarrow cleanup(g, live\_mask \setminus \{e\})$ 
7:    $child \leftarrow solve\_component(g, child\_mask)$ 
8:    $c \leftarrow g.edges[e].color$ 
9:   if  $c = \text{blue}$  or  $c = \text{green}$  then
10:     $L.append(child)$ 
11:   end if
12:   if  $c = \text{red}$  or  $c = \text{green}$  then
13:     $R.append(child)$ 
14:   end if
15: end for
16:  $G \leftarrow game\_make(L, R)$ 
17:  $G \leftarrow game\_canonicalize(G)$ 
18:  $position\_cache[g, live\_mask] \leftarrow G$ 
19: return  $G$ 
```

The function first recursively solves all positions reachable in one move and constructs the value of the current position from their values. The recursion depth is bounded above by the number of edges, because every move removes exactly one edge and the game is finite.

The computation begins by checking the `position_cache` hash table. If the position is not found, the algorithm iterates over all live edges. For every edge, a new position is created by removing the edge, its value is computed recursively, and the result is inserted into the array of Left or Right options according to the edge color. A green edge is added to both arrays, which is why two independent conditions are used instead of `if-else`. After all edges have been processed, the game is constructed from the collected options by calling `game_make` (line 16), and the result is canonicalized. The local arrays L and R (line 4) are arrays of pointers of type `Game*`, whose lifetimes are limited to the function call. The `game_make` function therefore copies the pointers into a newly allocated structure on the heap. The function uses a shallow copy, which may raise the question of how such a procedure can create a heap-based position when the child nodes are not copied. The reason is that this copy is already used for the zero position, where $L_{count} = R_{count} = 0$; that position is the only one that is deep-copied. Because the game has finitely many moves, however, every sequence of moves eventually terminates at a zero position, that is, at a game that has been deep-copied.

As in the Red-Blue Hackenbush algorithm (Algorithm 4), dominated positions could be removed while another move is being appended to the L and R arrays. Unlike Red-Blue Hackenbush, however, this does not generally leave only positions with 0 or 1 elements, because some values may be incomparable (see Section 4.2). The new position would therefore have to be compared with multiple existing positions every time it was added to the L and R arrays. This additional complexity provides no advantage.

Canonicalization is performed immediately after each subposition is constructed, rather than only on the final game. If canonicalization were postponed, noncanonical intermediate results would be stored in `position_cache`, causing the table to fill much more quickly. The Hackenbush state space is so large that prematurely exhausting the table's free slots would make

it useless before the computation reached the result. Incremental canonicalization therefore reduces the algorithm’s memory requirements at the cost of greater computational overhead.

7.4 Memoization and Hash Tables

Memoization is most commonly an optimization technique used to accelerate programs. It is a process in which a function automatically stores the results of previous computations. Traditionally, this is implemented as a hash table that gradually stores input-result pairs [8].

The solver maintains five hash tables using linear probing for collision resolution.

The `position_cache` table stores Hackenbush positions that have already been solved, so that the same position does not have to be recomputed repeatedly. This table corresponds to the mapping

(BaseGraph, edge_mask_t)
↓
*Game

In Chapter 8, all algorithms are generalized to arbitrary combinatorial games, together with the form of this table. The generalized table corresponds to the mapping

(void *, void *)
↓
*Game

The `game_geq_cache` table memoizes the results of game comparisons with respect to the relation $\geq$. In practice, this is the largest cache because the $\geq$ operation is called very frequently during canonicalization (see Section 7.7). This table corresponds to the mapping

(*Game, *Game)
↓
{0, 1}

The `game_add_cache` table stores the results of disjunctive sums because the same game combinations occur repeatedly during computation. This table corresponds to the mapping

(*Game, *Game)
↓
*Game

The `game_canon_cache` table stores the results of game canonicalization. Canonicalization includes removing dominated and reversible moves and is computationally expensive, so it is advantageous to canonicalize a game at most once. This table corresponds to the mapping

*Game
↓
*Game

Additional Hash Tables

The `game_intern_cache` table is a special hash table without which a raw game **cannot** be correctly converted into a short game in this implementation. It is difficult to classify it as a conventional memoization technique because a solution without it, or with a full table, fails critically. Its task is to ensure that each game has exactly one unique representation in memory. This is why the type `Game` is explicitly represented as `*Game`, a pointer to a short game, allowing some operations to be implemented using address arithmetic.

This table corresponds to the mapping

*Game
↓
*Game

which assigns each structural representation of a game its canonical instance. If a game with the same structure already exists in the table, the existing instance is returned. Otherwise, the new game is inserted into the table.

This approach corresponds to the technique known as *hash-consing* and ensures that two structurally identical games are represented by the same pointer. Consequently, game equality can be tested in constant time by comparing pointers, while memory requirements are also reduced substantially.

Before insertion into `intern_cache`, Left and Right moves are sorted by pointer value, ensuring a stable key independent of the order in which the moves were added. If the table already contains a structurally identical node, its pointer is returned; otherwise, the new node is inserted.

7.5 Canonicalization and Its Consequences

In Section 7.3, Algorithm 5, line 17 contains the function `game_canonicalize()`, which expects a short game represented by the `Game` structure defined in Subsection 7.1.

Canonicalization removes two types of redundant moves. The first are *dominated moves*: a Left option L' is dominated if another Left option L'' exists such that $L'' \geq L'$. Left has no reason to play it because L'' gives at least as good a position. The definition is symmetric for Right options. The second type consists of *reversible moves*: a Left option L' is reversible if a Right reply $L'^R \leq G$ exists. After such a move, Right can respond immediately and reach a position no better for Left than the original game G . Such a move by Left can be replaced directly by the Left options following L'_R , shortening the game tree without changing its value.

Canonicalization runs in a loop until no change occurs. Passes removing dominated moves alternate with bypassing reversible moves. After every change, the loop must restart because bypassing a reversible move introduces new options that may themselves be dominated or reversible.

The Simplest Dyadic Number of Short Games in Canonical Form

If the result of canonicalization is a short game $G = \{G^L \mid G^R\}$ that is a number, meaning $\forall l \in G^L, \forall r \in G^R : l < r$, then a consequence of canonicalization is that the remaining bounds $G_{max}^L = \max G^L$ and $G_{min}^R = \min G^R$ are the closest possible, and the number of Left and Right options is either 0 or 1, namely the numbers G_{max}^L and G_{min}^R (partially justified in Section 6.1.1).

This has a direct consequence for computing the simplest dyadic number of short games. If the Left and Right option sets each have exactly one element, the value is simply their average:

$$G = \frac{G_{max}^L + G_{min}^R}{2}.$$

The library contains the function `get_dyadic_value` in `src/short-games/C/singletons.c`, which converts a short game into a conventional dyadic number. Section 5.1 described that numerical short games form a subgroup of the group of short games $\mathbb{G}$ under game addition. This subgroup is isomorphic to the group of dyadic rational numbers $\mathbb{D}$. I therefore identify it directly with $\mathbb{D}$ and write short-game values such as G_{max}^L and G_{min}^R directly as the corresponding dyadic numbers.

I remind the reader that I use the terminology from [10], where a number is a short game satisfying the condition described in 5.1. By contrast, a dyadic number is the ordinary type of number familiar from elementary mathematics.

If a number is in canonical form and both players have exactly one option, G_{max}^L and G_{min}^R , then conversion to a dyadic number requires only averaging G_{max}^L and G_{min}^R .

Příklad 7.1 (Converting a number in canonical form to a dyadic number).

$$\begin{aligned}\{G^L \mid G^R\} &= \{3 \mid 4\} \\ G_{max}^L &= 3, \quad G_{min}^R = 4 \\ G &= \frac{3+4}{2} = 3.5\end{aligned}$$

Algorithm 6 Canonicalization of a game G

Require: A game $G = \{G^L \mid G^R\}$, with all subpositions already in canonical form

Ensure: The canonical form of G

```

1: if  $G \in \text{canon\_cache}$  then return  $\text{canon\_cache}[G]$ 
2: end if
3: repeat
4:    $changed \leftarrow false$ 
5:   for each  $G^{L_i} \in G^L$  do ▷ Reversible Left moves
6:     for each  $G^{L_i R_j} \in G^{L_i R}$  do
7:       if  $G \geq G^{L_i R_j}$  then ▷  $G^{L_i R_j} \leq G$ 
8:         Replace  $G^{L_i}$  by all  $G^{L_i R_j L_k}$ 
9:          $changed \leftarrow true$ ; break
10:      end if
11:    end for
12:  end for
13:  for each  $G^{R_i} \in G^R$  do ▷ Reversible Right moves
14:    for each  $G^{R_i L_j} \in G^{R_i L}$  do
15:      if  $G^{R_i L_j} \geq G$  then ▷  $G^{R_i L_j} \geq G$ 
16:        Replace  $G^{R_i}$  by all  $G^{R_i L_j R_k}$ 
17:         $changed \leftarrow true$ ; break
18:      end if
19:    end for
20:  end for
21:   $G^L \leftarrow \{G^{L_i} : \nexists j \neq i, G^{L_j} \geq G^{L_i}\}$  ▷ Remove dominated Left moves
22:   $G^R \leftarrow \{G^{R_i} : \nexists j \neq i, G^{R_j} \leq G^{R_i}\}$  ▷ Remove dominated Right moves
23:  if  $G^L$  or  $G^R$  changed then  $changed \leftarrow true$ 
24:  end if
25: until  $\neg changed$ 
26:  $K \leftarrow \text{intern}(G)$ 
27:  $\text{canon\_cache}[G] \leftarrow K$  return  $K$ 

```

Canonicalization begins by bypassing reversible moves. Left wants to maximize the game and Right wants to minimize it, so if Left moves to G^{L_i} , it is rational for Right to reply by moving to a position that minimizes its value, namely when $G \geq G^{L_i R_j}$. After canonicalization, the result is passed to `game_intern_cache`, which returns an existing identical node or inserts a new one.

7.6 Splitting Hotpotch into Independent Components

The sum of games, formally defined in Section 4.3, permits a position to be divided into independent components, each solved separately, with the resulting values then added. Two Hotpotch positions are independent if they share no vertex except a possible common root, the ground. A move in one component therefore has no effect on the other.

The split is performed on the graph of live edges. Loops directly on the ground, that is, edges for which $u = v = 0$, are processed separately because they do not form an ordinary component with a vertex above the ground. The remaining components are found by breadth-first search (BFS) over the graph vertices. For every vertex not yet visited, its entire connected component is traversed and all live edges belonging to it are collected.

Algorithm 7 Splitting a position into independent components

Require: Graph g , live mask $live$

Ensure: A list of masks $[m_1, m_2, \dots, m_k]$ of independent components

```

1:  $components \leftarrow []$ 
2:  $visited[v] \leftarrow \mathbf{false}$  for all vertices  $v$ 
3: for each live edge  $e \in live$  do ▷ Special case: loop from ground to ground
4:   if  $g.edges[e].u = 0$  and  $g.edges[e].v = 0$  then
5:      $components.append(\{e\})$ 
6:      $live \leftarrow live \setminus \{e\}$ 
7:   end if
8: end for
9: for each vertex  $i \geq 1$  do ▷ BFS over vertices 1 through  $|V| - 1$ 
10:  if  $visited[i]$  then continue
11:  end if
12:   $queue[0] \leftarrow i$ ;  $head \leftarrow 0$ ;  $tail \leftarrow 1$ ;  $visited[i] \leftarrow \mathbf{true}$ ;  $mask \leftarrow \emptyset$ 
13:  while  $head < tail$  do
14:     $curr \leftarrow queue[head]$ ;  $head \leftarrow head + 1$ 
15:    for each live edge  $e \in live$  do
16:      if  $\neg(g.edges[e].u = curr \text{ or } g.edges[e].v = curr)$  then continue
17:      end if
18:       $mask \leftarrow mask \cup \{e\}$ 
19:       $neighbor \leftarrow$  the other endpoint of edge  $e$  incident with  $curr$ 
20:      if  $neighbor \neq 0$  and  $\neg visited[neighbor]$  then
21:         $visited[neighbor] \leftarrow \mathbf{true}$ ;  $queue[tail] \leftarrow neighbor$ ;  $tail \leftarrow tail + 1$ 
22:      end if
23:    end for
24:  end while
25:  if  $mask \neq \emptyset$  then
26:     $components.append(mask)$ ;  $live \leftarrow live \setminus mask$ 
27:  end if
28: end for
29: if  $live \neq \emptyset$  then ▷ Edges with no path to ground; should not occur after cleanup
30:    $components.append(live)$ 
31: end if return  $components$ 

```

The result is a list of masks, each corresponding to one independent component. The value of the complete position is then the disjunctive sum of the values of the individual components,

$$G = G_1 + G_2 + \dots + G_k,$$

computed by the function `game_add`.

7.7 Comparing Games: $G \geq H$

By Definition 4.4, $G \geq H$ if and only if

1. for every Right move G^R from G , $\neg(H \geq G^R)$ holds,
2. for every Left move H^L from H , $\neg(H^L \geq G)$ holds.

This definition is sufficient to determine the ordering and is simple enough that stack-management overhead in an iterative algorithm slows the entire process. Because the algorithm uses depth-first search (DFS), whose depth can in the worst case equal the number of edges in the game, stack overflow is unlikely and it is preferable to exploit the simplicity of a recursive algorithm. Game comparison is the most important part of canonicalization and guarantees correct execution of the computation. Every canonicalization pass uses comparisons, causing this function to be invoked far more frequently than the other helper functions.

Algorithm 8 Recursive comparison of games $G \geq H$

Require: Games G and H

Ensure: The truth value of the relation $G \geq H$

```

1: if  $G = H$  then return true
2: end if
3: if  $(G, H) \in \text{geq\_cache}$  then return  $\text{geq\_cache}[(G, H)]$ 
4: end if
5: for each Right move  $G^{R_i} \in G^R$  do
6:   if  $H \geq G^{R_i}$  then                                ▷ Violation of  $\neg(H \geq G^{R_i})$ 
7:      $\text{geq\_cache}[(G, H)] \leftarrow \text{false}$ 
8:     return false
9:   end if
10: end for
11: for each Left move  $H^{L_i} \in H^L$  do
12:   if  $H^{L_i} \geq G$  then                                ▷ Violation of  $\neg(H^{L_i} \geq G)$ 
13:      $\text{geq\_cache}[(G, H)] \leftarrow \text{false}$ 
14:     return false
15:   end if
16: end for
17:  $\text{geq\_cache}[(G, H)] \leftarrow \text{true}$  return true

```

Chapter 8

Generalizing the Short-Game Solver to Other Games

The previous chapter described an explicit Hackenbush algorithm. It can be observed, however, that the algorithm can be generalized to solve any short game by implementing only a few functions. With correct implementations of these functions, a game can be solved without further modification of the solver code. An additional Python library was also added and can be used when building the interface for a particular game. The approach is therefore modular.

To construct the following modular algorithm, the recursive conversion algorithm from Hotpotch to a short game was modified incrementally. The process is shown below.

Section 7.1 contains the brief observation that a combinatorial game represented as a short game, of type `*Game`, *contains no* information about the internal structure of the original game. Consequently, every part of the code using only this type is already modular and requires no further modification. What remains are all parts that use the internal structure of Hotpotch, or Hackenbush, and therefore operate on the `BaseGraph` type.

The problem is thus to create an interface of functions implemented by the specific combinatorial game and used by the function that converts the game to a short game.

For clarity, the complete recursive short-game conversion algorithm from 5 is reproduced below and then modified step by step until the final version is reached.

8.1 A More General Conversion from a Raw Game to a Short Game

Algorithm 9 Recursive computation of a component as a short game

Require: Graph g , mask of live edges $live_mask$

Ensure: A pointer to the short game G

```

1: if  $(g, live\_mask) \in position\_cache$  then
2:   return  $position\_cache[g, live\_mask]$ 
3: end if
4: Initialize arrays  $L \leftarrow []$ ,  $R \leftarrow []$ 
5: for each live edge  $e \in live\_mask$  do
6:    $child\_mask \leftarrow cleanup(g, live\_mask \setminus \{e\})$ 
7:    $child \leftarrow solve\_component(g, child\_mask)$ 
8:    $c \leftarrow g.edges[e].color$ 
9:   if  $c = \text{blue}$  or  $c = \text{green}$  then
10:     $L.append(child)$ 
11:   end if
12:   if  $c = \text{red}$  or  $c = \text{green}$  then
13:     $R.append(child)$ 
14:   end if
15: end for
16:  $G \leftarrow game\_make(L, R)$ 
17:  $G \leftarrow game\_canonicalize(G)$ 
18:  $position\_cache[g, live\_mask] \leftarrow G$ 
19: return  $G$ 

```

As already described, lines operating on the `*Game` type are already modular. The remaining lines operate on `BaseGraph` and `edge_mask_t`. The other lines are displayed less prominently for clarity.

Generalizing Move Execution

The best place to begin is the incompatibility at lines 6 and 7. What do these lines do in principle? The `cleanup` function receives the game itself, g , and the mask after destroying the edge at index e . Thus, these lines are responsible for executing a move, and the new position is represented by the mask $live_mask$. A general function should not need to know what such a move looks like or what position results from it. It is therefore appropriate to define a function that executes the move.

There is, however, one important difference from the explicit algorithm. When a move is made in Hotpotch, or Hackenbush, the raw-game graph is always the same for both players. This is not true in general. For example, in *Domineering* (see Section 8.2), a move by Left changes the board differently from a move by Right at the same location. The interface will therefore provide separate move functions for Left and Right.

The next question is what arguments should be passed to these functions. Such a function will certainly require the game description itself. In the explicit algorithm, this was the type `BaseGraph`, or `*BaseGraph`. Because this is now a general function, its type will be a void pointer, `void *`, renamed with `typedef` to `RawGame_t`. The function implementation simply casts it to the appropriate type.

Section 6.1 explained why a separate variable of type `edge_mask_t` is used. A move often changes only a small part of the complete game description. In Hotpotch, this is the `live_mask`, which records which edges have been destroyed. If this dynamic information were included in the static part of the raw game, `RawGame_t`, unnecessary copies of the raw game would be created during execution. It therefore makes sense to add another general type, `Position_t`, also represented as `void *`. The final argument is the index `e` of the possible move. The return type is again `Position_t`, representing the game position after the move. This yields the following functions:

```
1 Position_t do_move_left(RawGame_t raw_game, Position_t position, int e);
2 Position_t do_move_right(RawGame_t raw_game, Position_t position, int e);
```

Listing 8.1: Functions for moves by Left and Right

Generalizing the Query of Whether a Move Can Be Made

Lines 8, 9, and 12 query the game rules to determine whether a move is possible in the current position. As before, these queries are split into two functions, because, for example, a query concerning a red Hackenbush edge has a different answer for each player. The functions certainly require the raw-game description of type `RawGame_t`. Returning to the Hotpotch example, an already destroyed edge cannot be destroyed again, so the functions must also receive information about the current position of type `Position_t`. Another argument is the possible move index `e`. Only a Boolean answer is required, so `int` is sufficient as the return type. This gives the following additional functions:

```
1 int can_left_move(RawGame_t raw_game, Position_t position, int e);
2 int can_right_move(RawGame_t raw_game, Position_t position, int e);
```

Listing 8.2: Functions querying whether Left or Right can move in a position

Generalizing Position Hashing

The problem on lines 1, 2, and 18 is that the general algorithm does not know how to hash such a game, so memoization could not be used. A design decision is required as to whether the hashing algorithm should also use information about the raw game. Omitting the raw game from hashing would mean that data from a hash table constructed during a previous conversion could not safely be reused. For example, in Hotpotch the mask recording whether an edge is destroyed *does not contain* information about its color. Different graphs may therefore have the same mask, so the same table cannot be used for different graphs. I chose to hash together with the raw-game description.

Another argument is naturally the position of type `Position_t`, together with the hashed move index `e`. This hash function is applied across all available moves, determined using `can_left_move` and `can_right_move`. The algorithm also needs a way to obtain the total number of moves on line 5. Another function must therefore be implemented to provide this information. This adds the following functions:

```
1 uint64_t hash_raw_game_position(RawGame_t raw_game, Position_t position, int e);
2 int num_moves(RawGame_t raw_game);
```

Listing 8.3: Function for obtaining the total number of moves and the position hash function

Generalizing the Split into Components

The final, less prominent problem is splitting the raw game into independent components. As described in Section 7.6, the task is to divide the game in its current position into smaller independent positions whose values are added. There is little difference from the previous approach, except that the implemented function must allocate its own position array because the general solver does not know the size of this array. This produces the final interface function:

```
1 int get_independent_components(RawGame_t raw_game, Position_t position,
2                               Position_t *sub_masks[]);
```

Listing 8.4: Function for obtaining an array of independent positions

The resulting algorithm is as follows.

Algorithm 10 A more general recursive conversion from a raw game to a short game

Require: Raw game *raw_game*, position *position*

Ensure: A pointer to the short game *G*

```
1: if (raw_game, position) ∈ position_cache then
2:   return position_cache[raw_game, position]
3: end if
4: Initialize arrays L ← [], R ← []
5: for e = 0, 1, ..., num_moves(raw_game) − 1 do
6:   if can_left_move(raw_game, position, e) then
7:     child_position ← do_move_left(raw_game, position, e)
8:     child ← solve_component(raw_game, child_position)
9:     L.append(child)
10:  end if
11:  if can_right_move(raw_game, position, e) then
12:    child_position ← do_move_right(raw_game, position, e)
13:    child ← solve_component(raw_game, child_position)
14:    R.append(child)
15:  end if
16: end for
17: G ← game_make(L, R)
18: G ← game_canonicalize(G)
19: position_cache[raw_game, position] ← G
20: return G
```

The algorithm no longer operates on a specific Hackenbush structure. For each move index, it merely asks whether the move is legal for Left or Right. If it is, the corresponding function `do_move_left` or `do_move_right` creates a new position. This position is recursively converted into a short game and inserted among the Left or Right options.

The construction of the game $\{L \mid R\}$ therefore remains the same for all short games. Differences between specific games are moved into the interface functions: `can_left_move`, `can_right_move`, `do_move_left`, `do_move_right`, `hash_raw_game_position`, `num_moves`, and `get_independent_components`.

8.2 Domineering

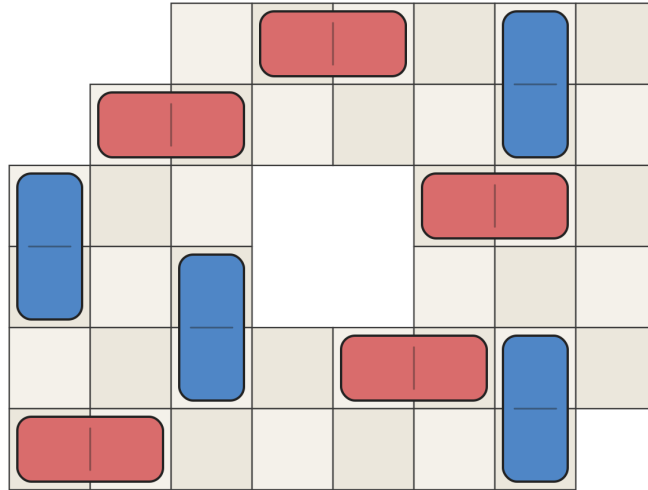


Figure 8.1: An example of an irregular Domineering position.

The combinatorial game Domineering is usually played on a rectangular grid, for example on a board of size $m \times n$, but in general the board may have any shape. Both players place 1×2 dominoes on empty squares. Left places dominoes only vertically, while Right places them only horizontally. A domino may not overlap an already occupied square and must lie entirely within the board. Under the normal-play convention, the player who can no longer make a move loses.

Domineering satisfies the conditions of a short game. The number of squares on the board is finite, and every move occupies exactly two previously empty squares. No move can therefore return the game to an earlier position, and every play ends after finitely many moves.

Basic Description of the Domineering Implementation

In the implementation, the raw game is represented by the board dimensions. The position itself is stored as a bit mask of occupied squares. The bit at position $i = y \cdot w + x$ determines whether the square with coordinates (x, y) is occupied.

The structures used to represent the game are:

```
1 // src/short-games/C/games/domineering/domineering.h
2 typedef struct {
3     uint8_t width;
4     uint8_t height;
5 } DomineeringBoard;
6
7 typedef struct {
8     uint64_t occupied_mask;
9 } DomineeringPosition;
```

Listing 8.5: Representation of the game Domineering.

A conversion from a raw Hackenbush game to a short game already existed, so it would not make sense to test the new method on the same game. To verify that the new algorithm works and to demonstrate the library’s functionality, two simple games were selected: Domineering 8.2 and Toads and Frogs [1, p. 12]. These games implement the functions from Section 8.1. A simple text interface in Python was also created for them.

8.3 File Structure

The thesis contains two different computational methods. The first method uses only the dyadic numbers from Chapter 5 to compute a position. The second method uses the structure of short games.

Files for the Dyadic-Number Method

All files for this method are located in the `src/dyadics` directory, which is divided into a library part, `src/dyadics/C`, and a graphical-interface part, `src/dyadics/python`. The library is written in C and contains four translation units:

1. `raw_game.c` and `raw_game.h`,
2. `dyadics.c` and `dyadics.h`,
3. `hash_game.c` and `hash_game.h`,
4. `utils.c` and `utils.h`.

The `raw_game` translation unit contains the definitions and implementations of functions responsible for processing the raw game. It includes, for example, the definition of the `BaseGraph` structure and the implementation of `cleanup_position`, which returns the position after an edge has been destroyed.

The `dyadics` unit contains functions responsible for computing dyadic numbers. It includes the definition of the dyadic-number structure and implementations of Algorithms 1, 2, and 3.

The `hash_game` unit contains implementations of position-hashing functions, and the `utils` unit contains helper functions such as `floor` and `ceil`.

Files for the Short-Game Method

All files for this method are located in `src/short-games`, which is similarly divided into a library part, `src/short-games/C`, and the Hotpotch graphical interface, `src/short-games/python`. The library additionally contains implementations of two other combinatorial games for which a text interface was created in Python: `domineering.py`, based on Domineering, and `toads_and_frogs.py`, based on Toads and Frogs as introduced in [1, p. 12]. Additional combinatorial games were added both to confirm that this solution approach works in principle and to demonstrate how the library can be used further. The Hotpotch graphical interface also contains a basic short-game calculator.

New combinatorial games are added as follows. The directory `src/short-games/C/shared` contains the file `raw_game.h`, whose functions must be implemented by the game. These are precisely the functions described in Chapter 8.1. The implementation of this file, together with the other files of the game, is added to `src/short-games/C/games/<game_name>`.

It is important to state that the general solver interacts with an implemented game only through `raw_game.h`. All header files shared by the implemented game and the general solver, including `raw_game.h`, are located in `src/short-games/C/shared`. This directory also contains files such as `stack.h` and `darray.h`.

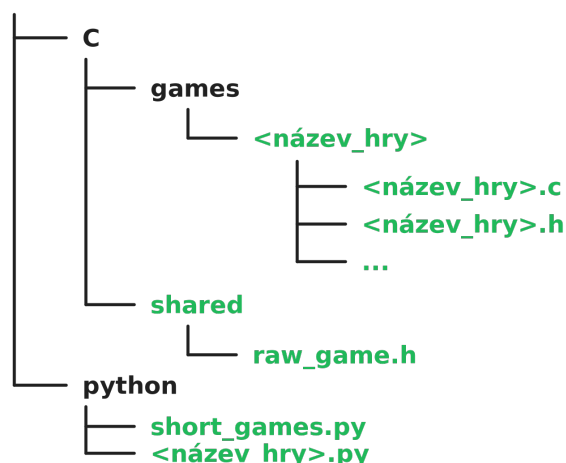


Figure 8.2: The directory and file tree, with the parts used by a newly implemented combinatorial game highlighted in green.

The notation `<game_name>` is a placeholder for the specific implemented game. Three games have already been implemented under the `games` directory, in the directories `hotpotch`, `domineering`, and `toad_and_frogs`. Uniform naming is *not* required, but is recommended. The program is built with `make`; if directory and file names are uniform and located correctly, it is not necessary to build the game separately from the rest of the program, and the `short_games.py` module automatically finds the compiled library.

The most important units of this library are

1. `short_game.c` and `short_game.h`,
2. `singletons.c` and `singletons.h`,
3. `position_cache.c` and `position_cache.h`,
4. `game_canon_cache.c` and `game_canon_cache.h`,
5. `game_intern_cache.c` and `game_intern_cache.h`,
6. `game_operations_cache.c` and `game_operations_cache.h`.

The main unit is `short_game`, which contains implementations of the most important operations, namely *comparison*, *addition*, *canonicalization*, and especially the function for converting a raw game into a short game.

The `singletons` unit contains references to important short games, such as the values 1, *, $\uparrow$, and others. It also contains helper functions, for example a function for converting a short game into a symbolic representation and a function for negating a game. These are practical functions that are not essential in principle.

The `position_cache` unit handles memoization of the raw game. Its name differs from those of the other tables because this table operates on a raw game and therefore requires more game-specific handling to keep the program modular.

The units `game_canon_cache`, `game_intern_cache`, and `game_operations_cache` use the type `Game *` and provide conventional memoization. The `game_operations_cache` contains two separate tables, one for addition and one for comparison.

The Hotpotch Graphical Interface

The Hotpotch graphical interface contains several extensions that are relevant to the content of this thesis or were requested by the supervisor.

1. Saving and loading game graphs:

The first extension is the ability to save and open game graphs. All graphs used in the thesis are included in the source code so that readers can try them. A game graph is stored in JSON format using functions in `hb_io.py`.

2. Position analysis:

Position analysis graphically displays dominated and reversible moves for the player to move. When analysis is enabled, each edge contains one of the following annotations.

- (a) $\langle i \rangle$ denotes the index of the edge.
- (b) $\langle i_1 \rangle R \text{ by } \langle i_2 \rangle$ means that the edge with index $\langle i_1 \rangle$ is reversible through the move with index $\langle i_2 \rangle$.
- (c) $\langle i_1 \rangle D \text{ by } \langle i_2 \rangle$ means that the edge with index $\langle i_1 \rangle$ is dominated by the move with index $\langle i_2 \rangle$.

This graphical position analysis is already used in Figures 4.4 and 4.5. The relevant methods are implemented in the `EduBubble` class.

3. Show hints:

Show hints graphically displays the best moves in the position, marked by a white aura around the corresponding edge. Hovering over any edge also displays the game value after that edge is destroyed.

4. Play against the bot:

The player's color is the color of the player currently to move. After the human player makes a move, the bot automatically plays one of the best available moves.

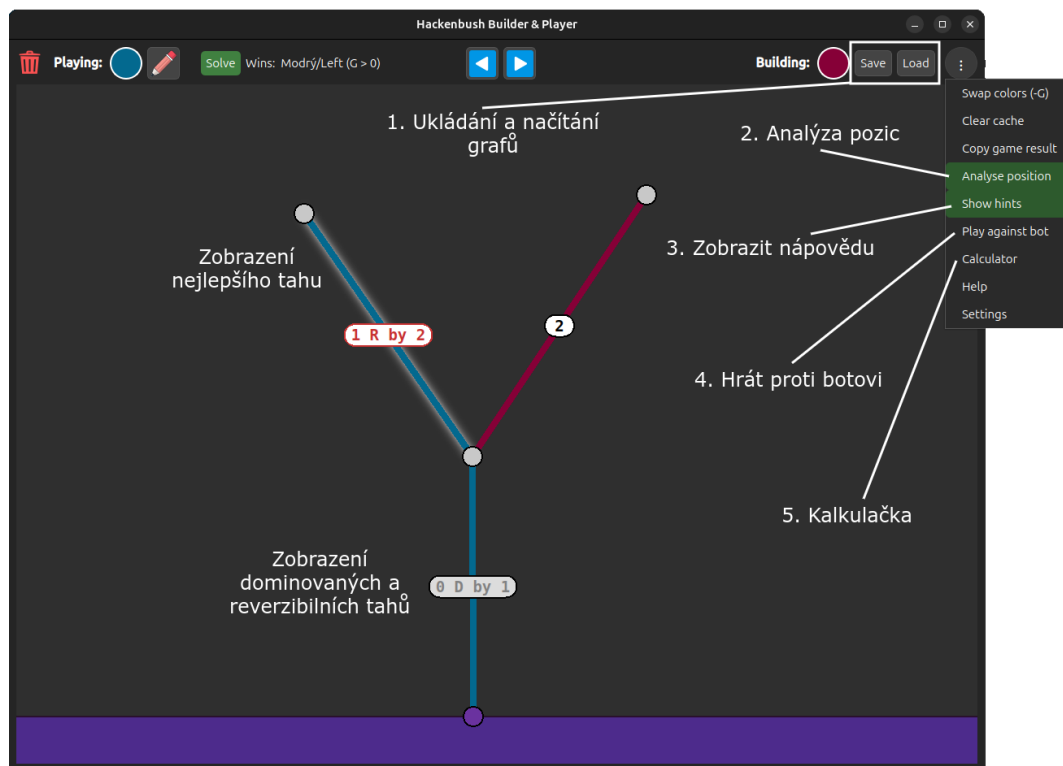


Figure 8.3: The Hotpotch graphical interface with explanations of the extensions.

5. Calculator:

The calculator is a more complex extension of this thesis. It supports basic algebraic operations on short games, together with short-game negation, short-game canonicalization, and a query determining whether a given short game is infinitesimal. The motivation for implementing it was the need to identify patterns in various sums and forms of short games while developing the function that converts a short game into a symbolic representation.

Calculator ← Back

Syntax: 0 1 -2 1/2 -3/4 * *3 1 ↓ 2↑ 2^ 3↓ 3v 1* 1* {0,*|1}

UNARY OPERATION

$\{\{\{0|1\}|1\}|\{\{\{0|1\}|1\}|\{\{\{0|1\}|1\}|\{\{0|1\}|1\}\}\}\}$ Canonization =

Result: $\{3|2+3\}$ ☐ raw

BINARY OPERATION

2 + ↓ $\{1,\{-2|*3\}|3^\wedge\}$ =

Result: $\{3|2+3\}$ ☐ raw

Note: raw output formats only defined number, which is 0 = $\{|\}$.

Figure 8.4: The graphical interface of the short-game calculator.

Chapter 9

Comparison of Approaches and Further Extensions

The thesis implements two different methods for evaluating Hackenbush positions. The first works with dyadic numbers and is intended only for Red-Blue Hackenbush. The second uses the general structure of short games and also handles Hotpotch, the variant with green edges.

Comparison of the Two Approaches

The dyadic-number approach has two main advantages. The value of a position is directly a number, so the result can be compared with zero without further processing. Removing dominated moves is trivial because dyadic numbers are linearly ordered, and the maximum Left option and minimum Right option always exist. The obvious disadvantage is that the method is restricted to Hackenbush. The file `src/short-games/python/scripts/test_hackenbush.py` contains a simple test of both approaches, which clearly shows that the dyadic-number method is faster. Owing to its simplicity, this method is also suitable for educational purposes.

The short-game approach is more general. It works for any combinatorial game that implements the interface described in Chapter 8. The cost of this generality is greater memory usage and computational complexity. An advantage of this approach is its straightforward extensibility and the ability to work directly with short games. It is also possible to provide an empty combinatorial-game implementation and use only the Python class `ShortGameSolver`, located in `short_games.py`, as a *Python short-game module*.

Table 9.1 summarizes the main differences.

Table 9.1: Comparison of the two approaches.

	Dyadic numbers	Short games
Supported combinatorial games	Hackenbush	arbitrary
Result type	$\frac{m}{2^n}$	arbitrary short game
Memory requirements	lower	higher
Time requirements	lower	higher
Interface generality	specific	modular

Possible Extensions

The most direct extension would be to modify the dyadic-number method so that it is as modular as the general short-game solver. At present, this method is tightly coupled to the Hackenbush data representation. The principle of computation with dyadic numbers could also be used for other combinatorial games whose values are always numbers. An interface analogous to the one in Chapter 8 could therefore be created, but with a return value of type `Dyadic` instead of `Game` *.

Another possibility is to add further combinatorial games to the general short-game solver. The library already contains an interface that permits a new game to be added by implementing several functions.

The graphical interface could provide a direct option for exporting the current position to an image. Functions for creating images already exist in the application, but they are currently used only to generate figures for this thesis.

The short-game calculator could be extended with a more general expression parser. The current version supports basic work with short games but does not allow convenient notation of more complex expressions, such as sums, differences, or nested operations on a single line.

A separate open problem is converting a short game into the simplest possible symbolic form. The implementation currently recognizes fairly complex special values, for example $D+n \uparrow$ where $D \in \mathbb{D}$, $n \in \mathbb{N}$, or $n*$ where $n \in \mathbb{N}$, but a general conversion would need to recognize a broader class of values.

Another possibility is to divide the library into two independent parts. The first part would contain only operations on short games. The second would contain the combinatorial-game solver itself. This separation would simplify use of the library in cases where the user does not want to solve a specific game but only wants to work with short games as algebraic objects.

Chapter 10

Conclusion

The objective of the thesis was to design and implement a solver for Hotpotch that can convert a given position into its value in combinatorial game theory. The thesis first introduced the required mathematical framework: combinatorial games, Hackenbush, Hotpotch, short games, canonical form, and dyadic numbers.

Two different solution methods were created in the implementation part. The first method uses the fact that Hackenbush position values can be expressed as dyadic numbers. Consequently, it is not necessary to store the complete short-game tree, and it is sufficient to work only with a value of the form

$$\frac{m}{2^n}.$$

This method is simpler and faster, but applies only to the numerical case.

The second method converts a raw position into a general short game. This representation is necessary for Hotpotch because green edges create values that are not generally dyadic numbers. The implementation therefore contains operations on short games, especially comparison, addition, negation, and canonicalization.

The general solver was subsequently generalized so that it is not rigidly tied only to Hackenbush and Hotpotch. A specific game needs to implement only functions for determining possible moves, executing a move, hashing a position, and splitting a position into independent components. This approach was verified on Domineering and Toads and Frogs.

The thesis also includes a graphical interface for Hackenbush and Hotpotch. The interface allows positions to be created and loaded, displays a position's value, highlights recommended moves, and graphically shows dominated and reversible moves. A simple short-game calculator using the same library as the solver was also created.

The result of the thesis is therefore a functional implementation of two approaches to solving Hackenbush. The dyadic-number method demonstrates a simpler and faster solution for Hackenbush. The short-game method is more general and permits deeper analysis of combinatorial games.

Bibliography

- [1] BERLEKAMP, E. R.; CONWAY, J. H. and GUY, R. K. *Winning Ways for Your Mathematical Plays*. Secondth ed. Wellesley, Massachusetts: A K Peters, 2001. ISBN 1-56881-130-6.
- [2] BOUTON, C. L. Nim, A Game with a Complete Mathematical Theory. *Annals of Mathematics*, 1901–1902, vol. 3, 1/4, p. 35–39. Second Series. Available at: <https://www.jstor.org/stable/1967631>.
- [3] CALISTRATE, D. The Reduced Canonical Form of a Game. In: *Games of No Chance*. Mathematical Sciences Research Institute, 1996, vol. 29, p. 409–416. MSRI Publications. Available at: <https://library.slmath.org/books/Book29/files/cali.pdf>.
- [4] CHO, R. *Blue Red Hackenbush Spiders*. San José, California, 2021. Master’s thesis. San José State University. Available at: https://scholarworks.sjsu.edu/etd_theses/5226. [cit. 2026-04-30].
- [5] CONWAY, J. H. *On Numbers and Games*. London: Academic Press, 1976. ISBN 0-12-186350-6.
- [6] ELKIES, N. D. On Numbers and Endgames: Combinatorial Game Theory in Chess Endgames. In: NOWAKOWSKI, R. J., ed. *Games of No Chance*. Cambridge University Press, 1996, vol. 29, p. 135–150. MSRI Publications. Available at: <https://doi.org/10.48550/arXiv.math/9905198>. Available as arXiv:math/9905198.
- [7] KNUTH, D. E. *Surreal Numbers*. Addison-Wesley, 1974. ISBN 0-201-03812-9. Seventeenth printing, April 2006.
- [8] NORVIG, P. Techniques for Automatic Memoization with Applications to Context-Free Parsing. *Computational Linguistics*, 1991, vol. 17, no. 1, p. 91–98. Available at: <https://aclanthology.org/J91-1004/>.
- [9] SCHLEICHER, D. and STOLL, M. *An Introduction to Conway’s Games and Numbers* online. October 2002. Available at: <https://www.cs.cmu.edu/afs/cs/academic/class/15859-s05/www/lecture-notes/comb-games-notes.pdf>. [cit. 2026-04-30].
- [10] SIEGEL, A. N. *Combinatorial Game Theory*. American Mathematical Society, 2013. Graduate Studies in Mathematics. ISBN 978-0-8218-5190-6.

Appendix A

Installation and Execution Guide

Dependencies and Compilation

To run the application, a C compiler supporting the C11 standard and the `make` tool must be installed. Python version 3.10 or later is required.

First, the shared C library must be compiled. From the root directory of the project:

```
1 make lib
```

Listing A.1: Compiling the C library

Virtual Environment and Python Dependencies

Before running any Python application, it is recommended to create a virtual environment:

```
1 python3 -m venv venv
2 source venv/bin/activate # Linux
```

Listing A.2: Creating and activating a virtual environment

The basic dependencies are listed in the `requirements.txt` file in the root directory of the project. They can be installed using:

```
1 pip install -r requirements.txt
```

Listing A.3: Installing dependencies

Running the Applications

Hackenbush graphical user interface:

```
1 cd src/dyadics/python
2 python3 hackenbush.py
```

Listing A.4: Running Hackenbush

Hotpotch graphical user interface:

```
1 cd src/short-games/python
2 python3 hotpotch.py
```

Listing A.5: Running Hotpotch

Domineering text-based user interface:

```
1 cd src/short-games/python
2 python3 domineering.py
```

Listing A.6: Running Domineering

Toads and Frogs text-based user interface:

```
1 cd src/short-games/python
2 python3 toads_and_frogs.py
```

Listing A.7: Running Toads and Frogs

Creating Images of Positions

The script `src/short-games/python/scripts/create_game_image.py` is used to create images of game positions. The script allows precise control over the resulting image through the following variables:

- `output_path` - the path and name of the output file,
- `margin` - the size of the margin around the position boundary when the boundary size is determined automatically,
- `fixedX` - a fixed boundary width; the content is centered horizontally,
- `fixedY` - a fixed boundary height; the content is centered vertically,
- `finalX` - the resulting image is resized to this width regardless of the original boundary size,
- `finalY` - the resulting image is resized to this height regardless of the original boundary size,
- `include_bubbles` - when set to `True`, graphical indicators of dominated and reversible moves are displayed in the image, as described in Section 7.6.

All images of game positions used in this thesis are located in the directory `src/short-games/python/thesis/games`.